

[Nome dell'Università]
[Nome del Dipartimento/Facoltà]
[Corso di Laurea Magistrale in ...]

Replica e Studio di Ablation di MetaSTGAT

per il Controllo Semaforico Adattivo

Relatore

[Nome e Cognome]

Candidato

[Nome e Cognome]

Anno Accademico [20XX/20XX]

Sommario

Il controllo semaforico su una rete di intersezioni è un problema di coordinamento distribuito: la fase scelta a un incrocio influenza, con pochi secondi di ritardo, il traffico che raggiunge gli incroci vicini. Le soluzioni classiche, a tempi fissi o reattive sulla sola pressione locale (MaxPressure), non tengono conto di questa dipendenza spaziale in modo esplicito. MetaSTGAT (Wang et al., 2022) affronta il problema con un agente di reinforcement learning per intersezione, coordinato da una *graph attention network* i cui pesi sono generati dinamicamente da un modulo di meta-learning condizionato su caratteristiche locali della rete stradale.

Questo lavoro replica da zero l’architettura e la procedura di training di MetaSTGAT su un ambiente costruito su CITYFLOW, con reti a griglia di tre dimensioni (4×4 , 5×5 , 6×6) e traffico generato parametricamente: ogni configurazione include un’arteria a densità artificialmente maggiorata, usata per misurare l’equità del controllo tra direzioni di traffico diverse. Ogni differenza introdotta rispetto alla procedura originale, dalla ricompensa alla struttura del replay buffer, è isolata da un sistema di preset di ablation e motivata singolarmente invece di essere presentata come un insieme indistinto di miglioramenti.

I risultati sperimentali raccolti finora confrontano le baseline classiche (Fixed-Time, MaxPressure) con diverse varianti del modello proposto. Un risultato non atteso emerge dallo sweep sul peso α della penalità anti-starvation della ricompensa: rimuoverla del tutto ($\alpha = 0$) non solo fa collassare l’equità direzionale verso i valori di MaxPressure, ma peggiora anche il travel time medio e massimo rispetto alle varianti con $\alpha > 0$, escludendo un semplice compromesso tra equità e throughput. Il confronto tra il modello completo e la variante senza BPTT e burn-in sulla componente ricorrente conferma inoltre il contributo di questo meccanismo, con un effetto proporzionalmente maggiore sulla generalizzazione a topologie mai viste che sulle configurazioni di training.

Al momento della stesura, lo studio di ablation copre due dei sei preset proposti e l’estensione dell’architettura a meccanismi spaziali alternativi alla *graph attention* (una convoluzione grafica e un meccanismo di propagazione a lungo raggio

ispirato a SONAR) è in fase di progettazione: questo sommario e le conclusioni saranno aggiornati man mano che quel lavoro sarà completato.

Indice

Sommario	i
Notazione e Acronimi	vii
1 Introduzione	1
1.1 Contributo di questo lavoro	2
1.2 Struttura della tesi	3
2 Background	4
2.1 Reinforcement Learning	4
2.2 Graph Neural Network	6
2.3 Traffic Signal Control come problema	9
3 TSC e Modelli	13
3.1 Formalizzazione del problema	13
3.2 Ambiente di simulazione	15
3.2.1 Struttura della rete e mappatura delle fasi	16
3.2.2 Gestione dell'interverde	18
3.2.3 Campo visivo	19
3.2.4 Dataset e generazione parametrica	19
3.2.5 Semplificazioni dichiarate	22
3.3 Baseline classiche	24
3.4 Architettura MetaSTGAT	25
3.4.1 State Encoder duale	25
3.4.2 Meta-knowledge learner	26
3.4.3 Meta-GAT	26
3.4.4 Meta-LSTM	27
3.4.5 Testa Q-value	28
3.4.6 Dal paper al modello Pro: differenze motivate	28
3.5 Procedura di training e differenze dal paper originale	30

4	Confronto	32
4.1	Metriche di valutazione	32
4.2	Protocollo sperimentale	35
4.3	Risultati: baseline e peso anti-starvation	38
4.4	Equità direzionale: il ruolo di α	39
4.5	Studio di ablation: risultati preliminari	41

Elenco delle figure

3.1	Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.	16
3.2	Rappresentazione grafica delle 8 fasi di Tabella 3.1: le frecce indicano i movimenti abilitati da ciascuna fase.	17

Elenco delle tabelle

3.1	Le 8 fasi semaforiche e i movimenti che ciascuna abilita.	17
3.2	Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.4.	20
3.3	Le cinque fasce del profilo di addestramento <i>complete</i> (totale: ~ 4.700 veicoli).	21
3.4	Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo.	22
4.1	Preset di ablation e sottosistema isolato da ciascuno.	38
4.2	Travel time medio, massimo e percentuale di completamento, mediati sulle 3 configurazioni di training (in-distribuzione).	38
4.3	Come Tabella 4.2, mediata sulle 7 configurazioni di test (generalizzazione a topologia e densità mai viste in training).	39
4.4	Attesa massima direzionale (N/S e W/E), mediata sulle configurazioni di training e di test rispettivamente.	40
4.5	MetaSTGAT Pro ($\alpha = 0.5$) contro <code>ablation_temporal</code> (BPTT e burn-in disattivati), stesso α	41

Notazione e Acronimi

Acronimi

Acronimo	Significato
MDP	Markov Decision Process
RL	Reinforcement Learning
DQN	Deep Q-Network
PER	Prioritized Experience Replay
BPTT	Backpropagation Through Time
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
TSC	Traffic Signal Control
SMK	Spatial Meta-Knowledge (learner)
TMK	Temporal Meta-Knowledge (learner)
CS	Cooperation Spatial (modulo Meta-GAT)
CST	Cooperation Spatial-Temporal (modulo Meta-GAT)

Notazione principale

Simbolo	Significato
$\mathcal{S}, \mathcal{A}, P, R, \gamma$	Stati, azioni, transizione, ricompensa, sconto (MDP, §2.1)
π, V^π, Q^π	Politica, funzione valore-stato, funzione valore-azione
θ, θ^-	Pesi della rete online e della target network
τ	Coefficiente di soft update (Polyak averaging)
α (in RL)	Esponente di prioritizzazione del PER
α (in questo lavoro)	Peso della penalità anti-starvation nella ricompensa <code>custom</code> (§3.1)
ε	Probabilità di esplorazione (ε -greedy)
s_i^t, a_i^t, r_i^t	Stato, azione, ricompensa dell'intersezione i al tempo t
n_i^t, w_i^t, p_i^t	Veicoli per corsia, attesa massima per corsia, fase corrente (componenti di s_i^t)
d_0	Dimensione dello stato in ingresso (32 avanzato, 20 paper)
d_1	Dimensione nascosta del modello (default 64)
H, d_h	Numero di teste di attenzione e dimensione per testa ($d_h = d_1/H$)
e_i^t, e_j^t	Embedding temporale e spaziale prodotti dal Dual State Encoder
x_i^t, h_i^t, c_i^t	Output, stato nascosto e stato di cella del Meta-LSTM
$\text{SMK}(i), \text{TMK}(i)$	Meta-embedding spaziale e temporale dell'intersezione i
$z_{\text{CS}}(i), z_{\text{CST}}(i)$	Output dei moduli Meta-GAT spaziale e spazio-temporale
$\overline{tt}, tt_{\max}$	Travel time medio (ufficiale) e massimo
<code>wait_maxNS/EW</code>	Attesa massima direzionale (equità, §4.1)

Capitolo 1

Introduzione

Un incrocio semaforizzato stabilisce periodicamente quali flussi veicolari autorizzare e quali arrestare. Su una singola intersezione l'ottimizzazione del numero di veicoli che riescono ad attraversarla è banale: si concede la precedenza alla direzione con il maggior traffico in attesa. Tuttavia se si considera una rete composta da vari incroci e se si cerca inoltre di ottimizzare anche il tempo di attesa dei veicoli, il problema diventa molto più complesso. La fase selezionata da un incrocio influenza con un certo ritardo temporale il volume di veicoli in arrivo alle intersezioni adiacenti. Pertanto il controllo semaforico su rete si configura come un problema di coordinamento distribuito ancor prima che di ottimizzazione isolata.

Attualmente la stragrande maggioranza delle intersezioni è regolata da sistemi a tempo fisso. Tale approccio prevede una sequenza di fasi di durata predeterminata, calcolata una tantum sulla base di stime di traffico medie. Il sistema non reagisce quindi in alcun modo alle condizioni osservate in tempo reale. Al giorno d'oggi si possono comunque individuare alcuni casi in cui la scelta della fase semaforica viene influenzata dal traffico, come negli incroci che presentano un sensore di rilevamento di veicoli in prossimità del semaforo, grazie al quale per esempio si attiva il flusso sulle strade secondarie solo quando si presenta fisicamente un veicolo. Questi sistemi sono però molto limitati, sia dal punto di vista dei dati raccolti, sia dal punto di vista delle azioni eseguibili. La ricerca si è quindi concentrata verso lo studio di sistemi di controllo semaforico che potessero, almeno in parte, reagire più concretamente alle condizioni di traffico in tempo reale, necessitando però di una raccolta di dati più elevata. Al momento si possono trovare nel mondo e in Italia incroci che sfruttano i risultati di queste ricerche, ma la loro diffusione non è ancora ampia a causa di ostacoli economici, tecnici, burocratici e normativi.

Detto ciò, lo studio di questi approcci reattivi rimane ad ogni modo interessante e di potenziale beneficio. Uno degli algoritmi che hanno ottenuto maggiore popo-

larità in questo ambito è MAX-PRESSURE (Varaiya, 2013), il quale introduce una regola locale che seleziona a ogni istante la fase con la pressione istantanea più alta (differenza tra veicoli in ingresso e in uscita sulle corsie servite da quella fase). MAX-PRESSURE garantisce proprietà di stabilità della rete nel suo insieme, ma la sua natura puramente greedy non offre alcuna garanzia di equità. Un’intersezione posta su un’arteria ad alto scorrimento tende a favorire sistematicamente la direzione dominante, costringendo i flussi minoritari ad attese indefinitamente lunghe: una criticità che questo lavoro osserva empiricamente e discute nel Capitolo 3 (la nozione di equità è definita e discussa formalmente nel Capitolo 4, §4.1).

Il controllo appreso tramite *reinforcement learning* (RL) propone un paradigma differente a quello di MAX-PRESSURE: invece di una regola scritta a mano, una politica π viene addestrata a massimizzare una ricompensa che codifica l’obiettivo di rete (tipicamente throughput e tempo di viaggio) osservando ripetutamente le conseguenze delle proprie azioni in simulazione. Applicato a una singola intersezione, il problema è modellabile come un MDP standard; su scala di rete, assume la forma di un sistema multi-agente a osservabilità parziale, dove ogni nodo rileva esclusivamente il proprio intorno locale. Emerge quindi la necessità di un meccanismo per la condivisione delle informazioni tra agenti adiacenti. Le *graph neural network* (GNN) assolvono a questa funzione: rappresentando la rete stradale come un grafo, con le intersezioni come nodi e le strade come archi, l’informazione si propaga tra nodi vicini, permettendo a ogni intersezione di condizionare la propria decisione sullo stato dei suoi vicini immediati (e non) senza richiedere una politica centralizzata.

MetaSTGAT (Wang et al., 2022) combina questi aspetti con uno ulteriore: invece di condividere gli stessi pesi tra tutte le intersezioni della rete, un modulo di *meta-learning* genera dinamicamente i pesi di uno strato di attenzione e di uno strato ricorrente a partire da caratteristiche locali dell’intersezione, così che nodi diversi possano essere serviti da trasformazioni diverse pur restando un unico modello addestrato end-to-end. Gli autori originali riportano una riduzione del travel time medio rispetto a CoLight (Wei et al., 2019), un metodo di riferimento basato su *graph attention* ma con pesi condivisi. La combinazione di *attention* su grafo, ricorrenza e reinforcement learning non è isolata: varianti di questo framework dominano la letteratura recente (Wang et al., 2026; Sun et al., 2026; Hu et al., 2026).

1.1 Contributo di questo lavoro

Questa tesi replica da zero l’architettura e la procedura di addestramento di MetaSTGAT, conducendo uno studio ablazionale sistematico e confrontandone le prestazioni

con le baseline classiche del controllo semaforico. Nello specifico, i contributi sono:

- un ambiente di simulazione multi-intersezione costruito su CITYFLOW (Zhang et al., 2019), con reti stradali a griglia di dimensione 4×4 , 5×5 e 6×6 , otto fasi semaforiche per intersezione e una formulazione MDP dettagliata nel Capitolo 3;
- una generazione parametrica dei dati di traffico, con due livelli di densità (*flat* e *peak*); in ciascuna configurazione viene potenziato artificialmente il traffico su una riga della griglia simulando un’arteria stradale, in modo da verificare sistematicamente se un modello di controllo produce uno squilibrio direzionale quando il traffico non è simmetrico;
- un sistema di ablation che isola singolarmente ogni differenza tra l’implementazione proposta in questa tesi e la procedura descritta nell’articolo originale, organizzato in sei configurazioni che vanno dalla replica fedele del paper al modello più avanzato di questo lavoro;
- un protocollo di valutazione che riporta media e caso peggiore del tempo di viaggio dei veicoli, oltre alla massima attesa registrata da parte di un veicolo all’interno di un incrocio

Al momento della stesura di questo capitolo, l’estensione dell’architettura verso meccanismi spaziali alternativi alla *graph attention* (una convoluzione grafica standard, GCN, e un meccanismo di propagazione a lungo raggio ispirato a SONAR) è ancora in fase di progettazione e non è inclusa nei risultati qui presentati. Il Capitolo 3 ne introduce comunque la motivazione, e il confronto sperimentale a parità di resto dell’architettura è rimandato a un aggiornamento successivo di questo lavoro.

1.2 Struttura della tesi

Il Capitolo 2 introduce il background necessario a leggere i capitoli successivi senza assumere familiarità pregressa con nessuno dei tre campi coinvolti: reinforcement learning, graph neural network e controllo semaforico. Il Capitolo 3 formalizza il problema affrontato come MDP, descrive l’ambiente di simulazione e le baseline classiche, e presenta l’architettura di MetaSTGAT insieme a ogni differenza introdotta rispetto all’articolo originale, motivata caso per caso. Il Capitolo 4 descrive il protocollo sperimentale e riporta i risultati ottenuti, incluso lo studio di ablation. Il Capitolo ?? discute i risultati alla luce della domanda di ricerca, dichiara i limiti dello studio e indica il lavoro rimasto per la seconda parte del progetto.

Capitolo 2

Background

2.1 Reinforcement Learning

Un problema di decisione sequenziale in ambiente stocastico è formalizzato come processo decisionale di Markov (MDP) (Sutton and Barto, 2018), definito dalla tupla

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

dove $\mathcal{S}$ è lo spazio degli stati, $\mathcal{A}$ lo spazio delle azioni, $P(s' | s, a)$ la probabilità di transizione markoviana, $R(s, a)$ la ricompensa attesa e $\gamma \in [0, 1)$ il fattore di sconto, che pondera l'importanza delle ricompense future rispetto a quelle immediate. Un agente segue una politica $\pi(a | s)$, una distribuzione di probabilità sulle azioni condizionata allo stato corrente; il suo obiettivo è massimizzare il ritorno atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

ovvero la somma scontata delle ricompense future a partire dall'istante t . La funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$ misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione. Queste soddisfano l'equazione di Bellman, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.1)$$

Per una politica ottima π^* che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

Il Q-learning tabellare approssima Q^* calcolando iterativamente l'Equazione 2.2, ma richiede una tabella con tutte le coppie (s, a) , il che risulta inapplicabile per spazi di stato $\mathcal{S}$ continui o ad alta dimensionalità, come nel caso del controllo semaforico.

Deep Q-Network (DQN). Mnih et al. (2015) superano tale limite sostituendo la tabella con una rete neurale $Q(s, a; \theta)$. Inoltre, per mitigare l'instabilità dell'addestramento dovuta alla correlazione temporale dei campioni, introducono il *replay buffer*, ovvero una memoria da cui vengono campionati mini-batch di transizioni (s, a, r, s') in modo casuale, ripristinando l'assunzione di indipendenza necessaria per la discesa del gradiente stocastico.

Esplorazione ε -greedy. Un agente che seguisse sempre e solo l'azione a valore Q stimato più alto non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima per via di una mancata esplorazione. La strategia più comune per bilanciare sfruttamento (*exploitation*, scegliere l'azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ε -greedy: a ogni passo di decisione, con probabilità ε l'agente sceglie un'azione uniformemente a caso, e con probabilità $1 - \varepsilon$ sceglie $\arg\max_a Q(s, a; \theta)$. Il valore di ε tipicamente decresce nel corso dell'addestramento, da un valore iniziale vicino a 1, dunque da un'esplorazione quasi totale, verso un valore finale piccolo ma non nullo, dunque conducendo ad un comportamento quasi del tutto deterministico.

Estensioni usate in questo lavoro. Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza; la procedura di training descritta nel Capitolo 3 le adotta tutte, motivandole caso per caso rispetto all'articolo di riferimento:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell'azione dalla sua valutazione, usando la rete online per scegliere l'azione migliore e una *target network* $Q(\cdot; \theta^-)$ solo per valutarla. Tale rete viene aggiornata periodicamente con i pesi della rete online e viene usata per calcolare il target $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$, in particolare $y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$. Questo meccanismo evita il problema del *moving target*, il quale si verifica invece ottimizzando questa stima rispetto a parametri in continua variazione, e riduce il bias sistematico di sovrastima del DQN standard.
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento propor-

zionale all'errore di differenza temporale (*TD error*) δ_i , ovvero la discrepanza tra il ritorno futuro stimato e quello ottenuto. Assegnando a ciascuna transizione una priorità $p_i = |\delta_i| + \varepsilon$, la probabilità di campionamento della stessa è $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, con $\alpha \in [0, 1]$ iperparametro che ne controlla la dipendenza dall'errore. Per correggere il bias introdotto da un campionamento non uniforme, la loss viene pesata tramite coefficienti di *importance sampling* $w_i = (N \cdot P(i))^{-\beta}$, dove N è la dimensione del buffer e $\beta \in [0, 1]$ iperparametro dinamico che ne controlla il grado di correzione.

- La **Huber loss** (o *smooth L1*) sostituisce l'errore quadratico medio (MSE) con una funzione quadratica vicino allo zero e lineare lontano da esso, risultando meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta \left(|y - \hat{y}| - \frac{1}{2}\delta \right) & \text{altrimenti} \end{cases} \quad (2.3)$$

Tipicamente $\delta = 1$.

- Il **soft update** (*Polyak averaging*) sostituisce l'aggiornamento periodico discreto della *target network* con una media mobile esponenziale:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

con $\tau \ll 1$, stabilizzando le variazioni del target a ogni passo di ottimizzazione.

- Quando la rete Q include un modulo ricorrente (ad esempio una LSTM come nel nostro caso, vedi Capitolo 3), l'assunzione di Markov sulle singole transizioni campionate dal *replay buffer* viene invalidata. Infatti, lo stato ricorrente (h, c) usato durante il training viene inizializzato a zero, introducendo un disallineamento rispetto allo stato generato sequenzialmente durante la raccolta dati: un problema noto come *hidden state staleness*. Seguendo Kapturowski et al. (2019), campioniamo intere sequenze di lunghezza L e dedichiamo i primi passi della sequenza (*burn-in*) esclusivamente a ricostruire uno stato ricorrente plausibile, senza calcolare gradiente, prima di eseguire la *backpropagation through time* (BPTT) sui passi restanti.

2.2 Graph Neural Network

Numerosi domini applicativi generano dati privi di una struttura regolare (come le immagini) o sequenziale (come il testo). Questi dati si organizzano spesso in topo-

logie a grafo dove le relazioni spaziali veicolano informazione cruciale. Un esempio naturale di grafo è costituito da una rete stradale: le intersezioni rappresentano i nodi, mentre le strade che le collegano sono gli archi. Lo stato di un'intersezione (ad esempio, il numero di veicoli in attesa o la fase semaforica attiva) è influenzato da ciò che accade nelle intersezioni adiacenti. Le *Graph Neural Network* (GNN) estendono i paradigmi di convoluzione e attenzione a questi domini a grafo, implementando meccanismi di *message passing* tra nodi adiacenti.

Graph Convolutional Network (GCN). Kipf and Welling (2017) propongono la formulazione più basilare di propagazione su grafo tra quelle qui discusse: la rappresentazione di ciascun nodo si aggiorna mediando quelle dei vicini con un peso fisso, determinato unicamente dalla topologia locale, non dal contenuto delle rappresentazioni stesse. Indicando con $\hat{A} = A + I$ la matrice di adiacenza a cui è stato aggiunto un self-loop per ogni nodo (cosicché un nodo contribuisca anche al proprio aggiornamento) e con $\hat{D}$ la matrice diagonale dei gradi corrispondente ($\hat{D}_{ii} = \sum_j \hat{A}_{ij}$), la regola di propagazione di un singolo layer si scrive, in forma matriciale:

$$H^{(l+1)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}) \quad (2.4)$$

dove $H^{(l)} \in \mathbb{R}^{N \times d_l}$ raccoglie le rappresentazioni di tutti gli N nodi del grafo al layer l , $W^{(l)}$ è la matrice di pesi condivisa da ogni nodo e σ una non linearità puntuale. Per un singolo nodo i , la normalizzazione simmetrica $\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ equivale a pesare il contributo di ogni vicino j con $1/\sqrt{\hat{d}_i \hat{d}_j}$:

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_j}} W h_j \right) \quad (2.5)$$

un peso identico per ogni arco del vicinato di i , funzione solo del grado dei due nodi coinvolti e mai del loro contenuto. È, per costruzione, il termine di paragone più naturale rispetto a cui misurare cosa aggiunge un meccanismo di attenzione appreso come quello discusso di seguito.

Graph Attention Network (GAT). Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, specifico per ciascuna coppia di nodi collegati: il contributo di un vicino non dipende più soltanto dalla topologia, ma anche dal suo contenuto. Dato un nodo i con vicinato $\mathcal{N}(i)$, embedding

di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{ij} = \text{LeakyReLU}(\vec{a}^\top [Wh_i \parallel Wh_j]), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.6)$$

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \right) \quad (2.7)$$

dove $\parallel$ denota la concatenazione. Con K teste di attenzione indipendenti, gli output si concatenano nei layer intermedi e si mediano nell'ultimo, secondo lo schema multi-head introdotto per l'attenzione da Vaswani et al. (2017). A differenza della GCN, il coefficiente α_{ij} consente a un nodo di ponderare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: una proprietà rilevante per un'intersezione i cui vicini abbiano importanza marcatamente diversa, come nel caso di un'arteria principale contrapposta a una strada secondaria. La funzione di compatibilità additiva appena descritta, per quanto la più nota, non è l'unica possibile: il Capitolo 3 (Meta-GAT) ne impiega una variante strutturalmente diversa, con Query, Key e Value esplicite e distinte al posto della proiezione W condivisa, e un prodotto scalare al posto della concatenazione.

Reti spatio-temporali su grafo. Una rete stradale non è soltanto un grafo: è un grafo il cui stato (occupazione delle corsie, fase attiva) evolve nel tempo, e le due dimensioni risultano intrecciate, poiché il valore osservato a un nodo in un dato istante è correlato sia con i valori passati allo stesso nodo sia con quelli ai nodi limitrofi. La famiglia di modelli che combina un layer di propagazione sul grafo (GCN o GAT) con una componente ricorrente o convoluzionale sull'asse temporale prende il nome di *spatiotemporal graph neural network* (STGNN), un termine ormai consolidato nella letteratura sul forecasting di serie temporali multivariate correlate (Cini et al., 2025). MetaSTGAT, descritto nel dettaglio nel Capitolo 3, costituisce un esempio di questa famiglia applicata al controllo semaforico anziché alla previsione: un layer di attenzione grafica cattura la dipendenza spaziale tra intersezioni limitrofe, un layer ricorrente (LSTM) ne cattura l'evoluzione temporale, e i due vengono addestrati congiuntamente end-to-end.

Pesi generati da meta-learning. Sia la GCN sia la GAT, nella loro formulazione originale, condividono gli stessi pesi su tutti i nodi del grafo: un'assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in una rete stradale reale, la topologia locale di ciascun nodo (numero di strade entranti, presenza di un'arteria, posizione periferica o centrale) può differire sensibilmente da

nodo a nodo. Un *hypernetwork* affronta il problema generando i pesi di un layer non come parametri fissi appresi direttamente, bensì come output di una seconda rete condizionata sulle caratteristiche locali del nodo: il Capitolo 3 descrive nel dettaglio come MetaSTGAT applichi questo principio sia al layer di attenzione grafica sia al layer ricorrente.

Propagazione a lungo raggio. Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un arco nel grafo) per layer: raggiungere un vicino a distanza k richiede impilarne altrettanti, con un costo di parametri crescente e un rischio noto di *over-smoothing*, per il quale le rappresentazioni dei nodi divengono troppo omogenee. Meccanismi più recenti, tra cui **SONAR** (Trenta et al., 2025), affrontano il problema della propagazione a lungo raggio secondo un principio diverso, ispirato a sistemi dinamici oscillatori piuttosto che alla concatenazione sequenziale di layer: il dettaglio di questo meccanismo e la sua applicazione al controllo semaforico sono rimandati al Capitolo 3.

2.3 Traffic Signal Control come problema

Il controllo semaforico (*Traffic Signal Control*, TSC) si articola in tre approcci, in ordine crescente di complessità.

Controllo a tempi fissi. Una sequenza di fasi di durata predeterminata, calcolata su stime di traffico medie, si ripete ciclicamente senza mai osservare lo stato reale della rete. È un approccio robusto, in quanto non può fallire per un sensore guasto, ma per costruzione non reattivo: la stessa sequenza viene eseguita indipendentemente da congestioni anomale, incidenti o variazioni del traffico non modellate in fase di pianificazione.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo. MAX-PRESSURE (Varaiya, 2013) ne è l'esponente più studiato: modellando il movimento dei veicoli come una rete di code *store-and-forward*, definisce la pressione di una fase ϕ come la differenza tra veicoli sulle corsie che quella fase servirebbe in ingresso e in uscita, e sceglie sempre l'azione a pressione massima. Il risultato teorico centrale dell'articolo originale è che Max-Pressure è *massimamente stabilizzante*; ciò significa che se è possibile stabilizzare la rete, ovvero non far divergere la lunghezza delle code nel tempo, Max-Pressure la stabilizza. L'algoritmo garantisce questo risultato usando solo la lunghezza delle code sulle corsie adiacenti all'intersezione, e senza conoscere a priori il volume di domanda in

ingresso. Max-Pressure è tuttavia miope per costruzione: non pianifica alcun compromesso su un orizzonte temporale non immediato, e la condizione di stabilità non tiene in considerazione le attese tra le diverse direzioni servite da un'intersezione, un aspetto di equità che questo lavoro esamina esplicitamente (Capitolo 4).

Controllo appreso. L'approccio basato sul *reinforcement learning* consente l'ottimizzazione diretta di metriche di rete globali (es. *travel time* o *throughput*) sostituendo euristiche come la pressione istantanea, al netto del costo computazionale dell'addestramento simulato. Applicato a una rete di più intersezioni, il problema si formalizza come un Processo Decisionale di Markov decentralizzato a osservabilità parziale (Dec-POMDP): ogni intersezione osserva solo il proprio stato locale, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti alterando il flusso di veicoli che le attraversano.

Architetture di coordinamento. Esistono tre strategie distinte che affrontano la dipendenza tra intersezioni vicine. Un agente centralizzato che decide le azioni di tutta la rete con un'unica politica elimina il problema per costruzione, ma lo spazio delle azioni congiunte cresce esponenzialmente con il numero di intersezioni ed è impraticabile oltre poche decine di nodi. All'estremo opposto, un agente indipendente per intersezione, che non condivide alcuna informazione con i propri vicini; questo approccio scala senza difficoltà ma tratta ogni intersezione come se fosse isolata, ignorando che le sue azioni si propagano alla rete circostante. La via intermedia, adottata anche da questo lavoro, fa comunicare agenti indipendenti condividendo informazione locale. La difficoltà, discussa da [Wei et al. \(2019\)](#), risiede nel fatto che la trasmissione dello stato dei vicini in una sequenza predefinita impone a tutte le intersezioni della rete gli stessi pesi interpretativi per ciascuna posizione della lista. Questo approccio non tiene conto della reale importanza che ogni specifico nodo adiacente può rivestire per le diverse intersezioni del sistema. Una GNN evita il problema: pesando il contributo di ogni vicino in base al suo contenuto invece che alla sua posizione in un elenco, permette a un unico insieme di pesi condiviso di generalizzare tra intersezioni con topologie locali diverse, come quelle della rete adottata in questo lavoro (Capitolo 3).

Coordinamento oltre l'adiacenza fisica. Il grafo usato da MetaSTGAT e da questo lavoro coincide con l'adiacenza fisica della rete stradale: un'intersezione comunica solo con le intersezioni immediatamente collegate da una strada. Dalla letteratura si osserva che interazioni tra sole intersezioni adiacenti non siano sufficienti a catturare tutte le dinamiche di traffico e si propongono diverse soluzioni per ovviare a questo limite. In questo lavoro si è deciso di non affrontare questo tema, esploran-

dolo solo in parte nel Capitolo 4. Il problema resta comunque aperto e framework che combinano ingredienti proposti da MetaSTGAT, quali attenzione grafica, una componente ricorrente sul tempo e reinforcement learning per il controllo semaforico, continuano ad essere oggetto di ricerca.

Una tassonomia delle soluzioni recenti. La letteratura più recente sul reinforcement learning applicato al TSC classifica i metodi lungo assi in parte indipendenti da quelli discussi finora. Un primo asse distingue, all'interno del multi-agente, due architetture di coordinamento: l'*addestramento centralizzato con esecuzione decentralizzata* (CTDE), in cui gli agenti condividono informazione globale solo durante il training per poi decidere localmente, e l'*addestramento ed esecuzione entrambi decentralizzati* (DTDE), in cui l'informazione resta sempre e solo locale, in entrambe le fasi. Un secondo asse raccoglie le tecniche di apprendimento integrate nel ciclo RL, ciascuna introdotta per rispondere a un limite specifico (Xiao et al., 2025): reti neurali a grafo per ottenere una migliore coordinazione tra intersezioni, componenti ricorrenti per catturare la dipendenza tra passi successivi a livello temporale, transfer-learning per generalizzare rapidamente a scenari nuovi, meta-learning per adattarsi a condizioni operative diverse, reti generative avversarie per compensare dati di stato mancanti o incompleti. Nessuna di queste tecniche risolve da sola il problema: la letteratura le introduce quasi sempre in risposta a un limite preciso.

Dove si colloca MetaSTGAT. Il modello di questo lavoro è la combinazione di alcune delle tecniche appena citate. La sua è un'architettura DTDE con scambio locale di informazione, che affianca all'attenzione grafica (GAT) un componente ricorrente (LSTM) per catturare la dipendenza temporale oltre a quella spaziale, collocandosi nella famiglia delle reti spatio-temporali su grafo (STGNN); il modello sfrutta inoltre il meta-learning, con l'obiettivo di adattare dinamicamente la propria architettura alle condizioni della rete.

Simulazione. Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo. Affinchè i risultati siano informativi, serve un simulatore che riproduca la dinamica del traffico con un dettaglio sufficiente, pur mantenendo un costo computazionale accettabile per l'addestramento di centinaia di episodi. CITYFLOW (Zhang et al., 2019) è il simulatore scelto in questo lavoro: rispetto ad alternative più diffuse come SUMO (Lopez et al., 2018), privilegia la velocità di simulazione rispetto al realismo microscopico del singolo veicolo, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con precisione millimetrica la dinamica

di un incrocio specifico. Il Capitolo 3 descrive nel dettaglio come questo lavoro configura l'ambiente CityFlow.

Capitolo 3

TSC e Modelli

3.1 Formalizzazione del problema

Il Capitolo 2.1 introduce l'MDP come la tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ e il Capitolo 2.3 la sua estensione a una rete di I intersezioni come un Dec-POMDP a osservabilità locale: ogni intersezione i osserva solo il proprio stato, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti. Questa sezione istanzia ciascuna componente di quella tupla per il sistema di questo lavoro. Ad ogni istante t , l'intersezione i osserva uno stato $s_i^t \in \mathcal{S}$, sceglie un'azione $a_i^t \in \mathcal{A}$ e riceve una ricompensa r_i^t , mentre l'informazione viene propagata secondo la topologia del grafo stradale.

Stato ($\mathcal{S}$). Ogni intersezione ha 4 strade in ingresso, una per direzione cardinale, con 3 corsie per senso di marcia, per un totale di `N_LANES` = 12 corsie in ingresso e altrettante in uscita. A questa configurazione si aggiungono `N_PHASES` = 8 fasi semaforiche selezionabili. Lo stato è la concatenazione di tre componenti:

$$s_i^t = [n_i^t \parallel w_i^t \parallel p_i^t] \in \mathbb{R}^{d_0} \quad (3.1)$$

dove $n_i^t \in [0, 1]^{12}$ è il numero di veicoli per corsia in ingresso (normalizzato dividendo per 30 e poi limitato tra 0 e 1), $w_i^t \in [0, 1]^{12}$ è il vettore dei tempi di attesa dei veicoli da più tempo in corsia (normalizzato dividendo per 100 e poi limitato tra 0 e 1) e $p_i^t \in \{0, 1\}^8$ è la fase corrente in codifica one-hot. La componente w_i^t non è presente nella formulazione originale del paper, che usa $d_0 = 20$; la variante di questo lavoro con w_i^t inclusa porta $d_0 = 32$. Inoltre, nel presente studio, n_i^t conta solo i veicoli entro una distanza `VISION_CUTOFF_M` dal semaforo (più nel dettaglio in §3.2), mentre nell'articolo usato come riferimento la visibilità è sull'intera corsia.

Infine, per preservare l'invarianza dimensionale del vettore di stato $s_i^{(t)}$, le intersezioni periferiche con grado topologico inferiore a quattro vengono completate tramite *zero-padding*.

Azione ($\mathcal{A}$). $a_i^t \in \{0, \dots, 7\}$ seleziona una delle otto fasi che verranno descritte in §3.2; La transizione di "tutto-rosso" è gestita internamente dal simulatore e non appartiene allo spazio delle azioni. Un vincolo anti-starvation invalida la fase corrente come azione se è già stata selezionata due volte consecutive, forzando la rotazione:

$$\mathcal{A}_i^t = \{0, \dots, 7\} \setminus \{a_i^{t-1}\} \quad \text{se } a_i^{t-1} = a_i^{t-2}, \quad \mathcal{A}_i^t = \{0, \dots, 7\} \text{ altrimenti} \quad (3.2)$$

Questo vincolo si applica solo agli agenti RL, mentre MaxPressure resta fedele all'implementazione di riferimento. Il numero due non è arbitrario: nelle primissime fasi di training, quando la politica è ancora vicina al caso e non ha alcuna preferenza appresa, un agente senza questo vincolo tende a bloccarsi sulla prima fase che produce un ritorno anche solo leggermente positivo, ripetendola indefinitamente; vietare la ripetizione oltre la seconda volta consecutiva rompe questo ciclo senza impedire a una fase di restare attiva per un tempo ragionevole quando è davvero la scelta migliore per due passi di fila.

Transizione (P). Essendo l'algoritmo di addestramento (Double DQN) *model-free*, le probabilità di transizione $P(s' \mid s, a)$ non vengono modellate esplicitamente, ma campionate empiricamente. L'ambiente CITYFLOW è per natura deterministico, pertanto, fissati stato e azione, la transizione collassa su un singolo stato successivo con probabilità 1.

Fattore di sconto (γ). In questo lavoro viene fissato $\gamma = 0.85$ per ogni modello, senza alcuna ottimizzazione iperparametrica dedicata.

Ricompensa (R). Vengono proposte due modalità.

La modalità `custom`, introdotta in questo lavoro:

$$r_i^t = \text{clip} \left(\frac{-(\text{Incoming}_i^t - \text{Passed}_i^t) - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -2, 0 \right) \quad (3.3)$$

dove Incoming_i^t è il numero di veicoli entrati nella finestra visiva dell'intersezione al tempo t , Passed_i^t il numero di veicoli che hanno lasciato la finestra visiva tra t e $t+1$, MaxRedWait_i^t il tempo di attesa massimo tra tutti i veicoli che si trovano nelle corsie con segnale rosso in ingresso all'istante t , $\text{Penalty}_i^t = 50$ se $\text{Passed}_i^t =$

$0 \wedge \text{Incoming}_i^t > 0$ e 0 altrimenti, e $\alpha > 0$ l'iperparametro anti-starvation, con il quale si modula la penalizzazione per lo starvation. Il limite superiore del clip è 0 per costruzione, il limite inferiore (-2) è stato invece calibrato sperimentalmente nel corso di questo lavoro.

La modalità **paper** riproduce fedelmente Wang et al. (2022, Eq. 1):

$$r_i^t = -\frac{P_i^t}{100}, \quad P_i^t = \text{Incoming}_i^t - \text{Outgoing}_i^t \quad (3.4)$$

dove P_i^t è definito come la *pressione dell'intersezione* i al tempo t , ovvero la differenza tra il numero di veicoli in entrata all'intersezione al tempo t (Incoming_i^t) e il numero di veicoli in uscita dall'intersezione al tempo t (Outgoing_i^t). Il fattore di scala $1/100$ in entrambe le modalità è introdotto per stabilizzare i gradienti. Per il calcolo di tutte queste metriche introdotte ciascun modello fa riferimento, se impostata, alla distanza `VISION_CUTOFF_M`.

Topologia di Rete (G). Le I intersezioni sono rappresentate dai nodi di un grafo $G = (V, E)$. Un arco orientato $(i, j) \in E$ esiste se un segmento stradale connette fisicamente i a j . A ogni nodo è inoltre aggiunto un arco verso se stesso (*self-loop*), cosicché i layer di *graph attention* possano pesare tanto lo stato del nodo stesso quanto quello dei vicini. Sotto il profilo della generalizzazione spaziale, la presente architettura impone che lo stato $s_i^{(t)}$ e le *meta feature* processino unicamente dati locali, senza alcun accesso all'identità del nodo i all'interno della rete. Conseguentemente, la policy non può prendere decisioni in base alla posizione dell'incrocio o al suo ruolo all'interno della rete (es. questo incrocio rappresenta un'arteria), ma è costretta ad apprendere euristiche reattive alle condizioni locali osservate.

3.2 Ambiente di simulazione

L'ambiente di simulazione si basa su CITYFLOW (Zhang et al., 2019), un simulatore microscopico open source con motore in C++ e binding in Python, progettato specificamente per il controllo semaforico su larga scala tramite reinforcement learning. La necessità di un nuovo strumento, come argomentato dagli autori originali, nasce dai limiti prestazionali di SUMO (Lopez et al., 2018), il simulatore open source più diffuso in letteratura. SUMO risulta infatti computazionalmente proibitivo per l'addestramento di agenti RL su reti di grandi dimensioni e ad alta densità veicolare. CITYFLOW garantisce una velocità di simulazione circa 20 volte superiore rispetto a SUMO su griglie 30×30 , ma il compromesso è un dettaglio minore, il che si ri-

specchia su una dinamica veicolare semplificata. Tale trade-off risulta ottimale in questo contesto, dove il vincolo critico è l'esplorazione dello spazio delle policy su migliaia di episodi, piuttosto che la fedeltà microscopica della singola traiettoria.

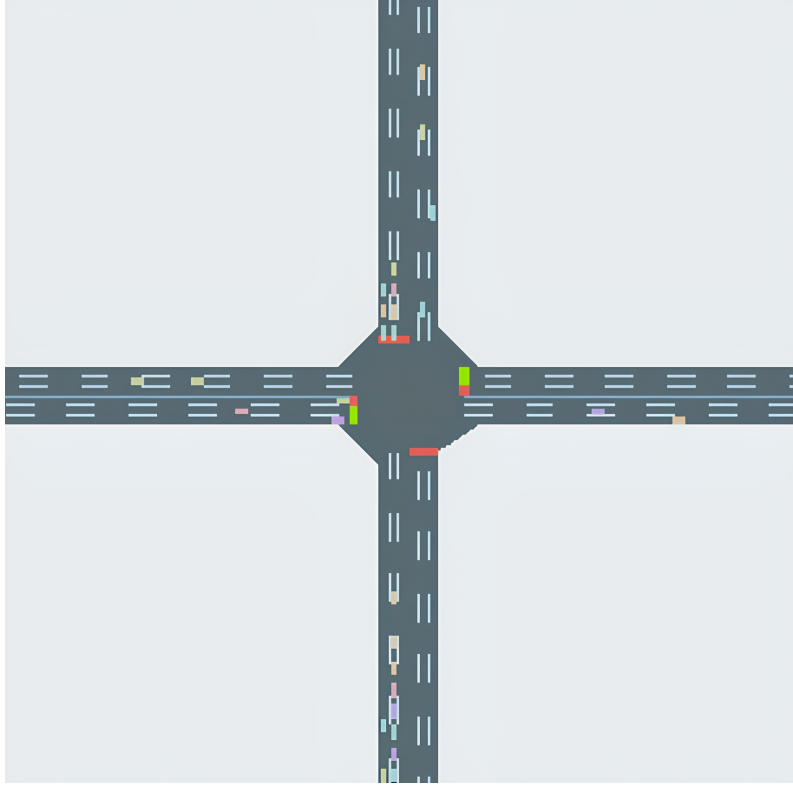


Figura 3.1: Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.

3.2.1 Struttura della rete e mappatura delle fasi

La topologia della rete è codificata in un file di configurazione (*roadnet*), che definisce l'insieme delle intersezioni (nodi) e dei segmenti stradali (archi) con le relative molteplicità di corsia. Per ogni intersezione il *roadnet* elenca le fasi semaforiche disponibili, ciascuna definita come l'insieme dei movimenti (coppie corsia di origine e di destinazione) abilitati in quella fase. L'azione dell'agente si riduce pertanto alla selezione dell'indice corrispondente alla fase da attivare. Ogni intersezione di questo lavoro modella $N_LANES = 12$ corsie in ingresso (quattro direzioni cardinali, tre corsie ciascuna: sinistra, dritto, destra) e $N_PHASES = 8$ fasi semaforiche selezionabili. La tabella 3.1 riporta la mappatura di tali fasi, adottando la nomenclatura N/S/E/W (*North/South/East/West*) e T/L (*Through*, dritto, e *Left*, sinistra).

Fase	Nome	Movimenti abilitati
0	NTST	Nord diritto+destra, Sud diritto+destra
1	NLSL	Nord sinistra, Sud sinistra
2	NTNL	Nord sinistra+diritto+destra
3	STSL	Sud sinistra+diritto+destra
4	WTET	Ovest diritto+destra, Est diritto+destra
5	WLEL	Ovest sinistra, Est sinistra
6	ETEL	Est sinistra+diritto+destra
7	WTWL	Ovest sinistra+diritto+destra

Tabella 3.1: Le 8 fasi semaforiche e i movimenti che ciascuna abilita.

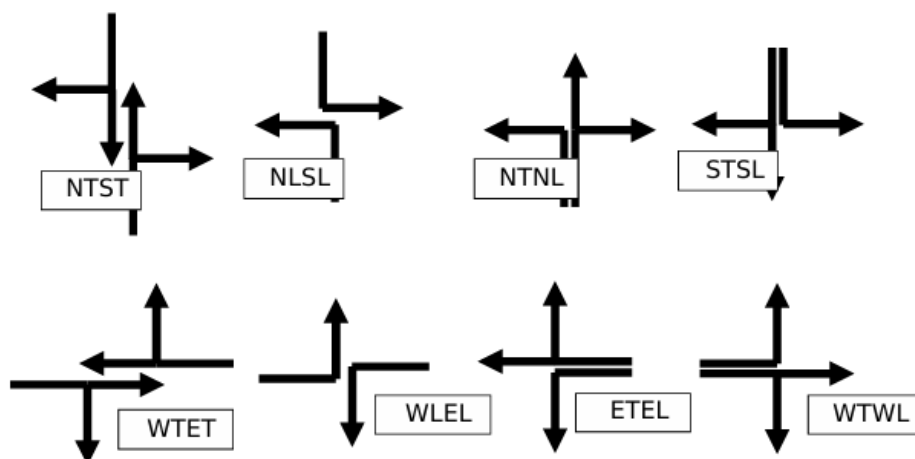


Figura 3.2: Rappresentazione grafica delle 8 fasi di Tabella 3.1: le frecce indicano i movimenti abilitati da ciascuna fase.

Il paper originale permette ai veicoli che svoltano a destra di muoversi liberamente, senza attendere il semaforo verde, a patto che non vi siano veicoli provenienti da altre direzioni. Al fine di prevenire una qualunque possibile interferenza tra diversi flussi e garantire un controllo più rigoroso, si è deciso di non adottare questo comportamento, permettendo la svolta a destra solo quando esplicitamente autorizzata dalla fase semaforica. L'insieme delle fasi sopra riportate rappresenta una configurazione tale da sfruttare al massimo lo spazio delle direzioni possibili, evitando al contempo conflitti tra le traiettorie dei veicoli.

3.2.2 Gestione dell'interverde

L'interverde è il periodo di tempo che intercorre tra la fine del verde di una fase semaforica e l'inizio del verde di quella successiva. Il suo scopo è quello di garantire la sicurezza stradale, permettendo ai veicoli che si trovano nell'intersezione di sgombrarla prima che venga attivata la fase successiva. Il motore di simulazione CITYFLOW non modella nativamente tale fase transitoria, nello specifico, non gestisce i semafori gialli e il conseguente passaggio sicuro tra due fasi consecutive; dunque l'interverde deve essere costruito esplicitamente. In questo lavoro, l'intervallo decisionale dell'agente (lo *step* di simulazione) ha una durata complessiva $\Delta t_{\text{step}} = 15$ s simulati, idealmente ripartita in tre sotto-fasi: verde ($\Delta t_{\text{green}} = 10$ s), giallo ($\Delta t_{\text{yellow}} = 3$ s) e "tutto-rosso" ($\Delta t_{\text{red}} = 2$ s). Per il limite di CITYFLOW sopra descritto e in questo studio accettato come compromesso, l'ambiente assimila il giallo al verde senza imporre modelli di decelerazione specifici, mantenendo pertanto i movimenti autorizzati dalla policy per una durata totale di $\Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s consecutivi. Al termine di questa finestra temporale, viene eseguita per $\Delta t_{\text{red}} = 2$ s una fase che non è presente tra le azioni selezionabili dal modello, ovvero il semaforo rosso in contemporanea su tutte le corsie. Questo intervallo di guardia è necessario per garantire lo svuotamento dell'intersezione prima del cambio di fase, così da prevenire collisioni. A questo punto lo step giunge al termine e il modello formula una nuova decisione, scegliendo quale delle 8 fasi implementare nel ciclo successivo.

È importante sottolineare che questa accortezza riduce di fatto il tempo effettivo utile di decisione del singolo agente di controllo semaforico, in quanto due dei quindici secondi di ogni step sono dedicati alla fase "tutto-rosso" e quindi da considerarsi tempo morto (rappresentando circa il 13% del tempo totale).

3.2.3 Campo visivo

Un contributo originale di questo studio consiste nell'introduzione di un campo visivo limitato per ciascun incrocio, per cui i dati raccolti da ogni intersezione circoscritte entro un raggio di visibilità d_{vis} . Le motivazioni alla base di questa implementazione sono da ricercare in due ragioni distinte: una di carattere pratico e una di carattere teorico. La prima mira a ridurre il gap tra simulazione e realtà, per il quale è essenziale limitare l'uso della strumentazione tecnica necessaria per la raccolta dei dati ad un'area il più circoscritta possibile. La seconda trova fondamento nella cinematica dei veicoli: ogni veicolo in questa simulazione raggiunge una velocità di percorrenza massima $v_{\text{max}} = 11.1$ m/s (circa 40 km/h) dettata dal limite di velocità imposto su ciascuna strada. Ciò implica che in un intervallo di tempo fissato $\Delta t = \Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s, un veicolo possa percorrere una distanza non superiore a

$$d_{\text{vis}} = v_{\text{max}} \cdot \Delta t \approx 144.3 \text{ m.}$$

Il calcolo esclude coerentemente i secondi di "tutto-rosso", in cui il deflusso è interrotto. Dunque la scelta è stata quella di introdurre un campo visivo ristretto anche per forzare l'agente RL a concentrarsi esclusivamente sui veicoli che una fase semaforica può effettivamente influenzare. Sebbene in questo lavoro il cutoff spaziale sia assunto omogeneo per l'intera rete, il codice che lo implementa tiene nativamente in considerazione eventuali casi di limiti di velocità o intervalli temporali variabili. Inoltre la riduzione del campo visivo non viene applicata se la strada risulta essere più breve del cutoff stesso. Questa limitazione percettiva è comunque riservata ai soli modelli di reinforcement learning di questo studio e non viene applicata alla baseline (come MaxPressure).

3.2.4 Dataset e generazione parametrica

Ogni roadnet e ogni flusso di traffico usato in questo lavoro è generato parametricamente e in modo interamente deterministico da uno script dedicato, senza alcun campionamento empirico. Questo approccio consente di avere un pieno controllo sulle variabili operative come la struttura della rete, la densità veicolare e presenza di arterie principali, oltre a garantire l'esatta riproducibilità degli scenari di simulazione, un requisito metodologico fondamentale per un equo confronto tra modelli di controllo semaforico.

Topologie e spaziature. L'ambiente di simulazione implementa tre topologie a griglia regolare di dimensioni 4×4 (16 intersezioni), 5×5 (25 intersezioni) e

6×6 (36 intersezioni), impiegate per l'addestramento e per valutare la capacità di generalizzazione dei modelli rispetto alla scala spaziale.

Un obiettivo centrale di questo lavoro consiste nell'indagare le capacità di coordinamento implicito apprese dalle diverse policy di controllo. In quest'ottica, assume particolare rilevanza la comparsa spontanea della così detta onda verde, uno standard strutturale nell'ingegneria del traffico contemporanea. La spaziatura tra incroci è pertanto di due tipi: calibrata per l'onda verde e non calibrata. In assenza di offset temporali, in quanto le transizioni di fase sono sincrone per tutti i nodi della rete, l'unico modo per ottenere un'onda verde, ovvero un flusso di veicoli che si propagano attraverso la rete senza fermarsi ai semafori, è calibrare la distanza fisica tra gli incroci sulla velocità di marcia e sulla durata del ciclo: alla velocità di riferimento $v_{\text{urb}} = 11.1 \text{ m/s}$, in un ciclo di $\Delta t_{\text{step}} = 15 \text{ s}$ viene percorsa una distanza pari a 166.5 m. Dunque la prima spaziatura proposta è proprio di questa misura, ma ne faremo riferimento con l'etichetta "100m". La spaziatura non calibrata invece è 100m più estesa e viene indicata con "200m".

Le roadnet non condividono tutte lo stesso ruolo. La griglia 4×4 con spaziatura "100m" compare sia in training sia in test, mentre le altre tre combinazioni topologia-spaziatura restano riservate esclusivamente al test e non vengono mai incontrate durante l'addestramento (Tabella 3.2). Questa separazione serve a isolare due assi di generalizzazione: le griglie 5×5 e 6×6 misurano la tenuta della policy su reti più estese di quella di addestramento, con più intersezioni da coordinare, mentre la spaziatura "200m" applicata alla griglia 4×4 misura la tenuta su una topologia mai vista con incroci più distanziati, priva della calibrazione per l'onda verde descritta sopra.

Griglia	Spaziatura	Utilizzo
4×4	100m	Train, Validation, Test
4×4	200m	Test
5×5	100m	Test
6×6	100m	Test

Tabella 3.2: Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.4.

Profili di traffico. Per ciascuna topologia sono stati generati tre profili di traffico: un flusso a intensità costante (*flat*), un profilo a gradino (*peak*, diviso in tre fasi temporali con un raddoppio del carico nel terzo centrale) e un profilo dinamico (*complete*) che simula il traffico di una giornata lavorativa (dettagliato nel seguito), impiegato esclusivamente in fase di addestramento. Ogni episodio ha una durata

di $T_{\max} = 1800$ s simulati, corrispondenti a $1800 \text{ s} / \Delta t_{\text{step}} = 120$ cicli decisionali per agente.

I volumi dei veicoli immessi nella rete per ciascuna configurazione topologica sono stati determinati tramite una calibrazione empirica, così da garantire un livello di saturazione comparabile tra reti di dimensioni diverse e quindi una difficoltà del task di controllo invariata rispetto la scala topologica. In particolare la densità di ogni topologia è stata bilanciata affinché il *throughput* ottenuto dalla baseline MAX-PRESSURE risultasse omogeneo (90%–94%).

Profilo di addestramento *complete*. A differenza degli scenari di validazione e test, caratterizzati da densità costanti o a singolo picco, il profilo di addestramento sintetizza l’evoluzione della domanda veicolare di un’intera giornata solare (24 ore) all’interno di un singolo episodio di 1800 s. Come dettagliato nella Tabella 3.3, il flusso è diviso in cinque fasce a intensità variabile: due fasce a basso carico (assimilabili agli orari notturni), due picchi di massima domanda (flussi di traffico di inizio e fine turno lavorativo) e un intervallo di raccordo centrale a regime moderatamente sostenuto. Questa marcata variazione di domanda veicolare all’interno di un singolo episodio è funzionale poiché costringe il modello ad adattare dinamicamente il piano di controllo, senza convergere verso una policy specializzata soltanto su un unico flusso di traffico.

Fascia	Intervallo simulato	Step di decisione	Veicoli/secondo
Flat basso	0-525s	0-35	1.2
Peak basso	525-675s	35-45	3.5
Flat alto	675-1200s	45-80	2.5
Peak alto	1200-1350s	80-90	4.2
Flat basso	1350-1800s	90-120	1.2

Tabella 3.3: Le cinque fasce del profilo di addestramento *complete* (totale: ~ 4.700 veicoli).

Stocasticità e sbilanciamento direzionale. Per mitigare il rischio di *overfitting* e garantire la robustezza del modello, la generazione del profilo di addestramento integra due accorgimenti.

Innanzitutto, considerata la natura deterministica di CITYFLOW, addestrare un modello su una singola configurazione esporrebbe l’agente all’apprendimento mnemonico di un’esatta sequenza di veicoli. Per prevenire questa eventualità, si generano con diversi *seed* tre profili di traffico differenti, in modo da perturbare la distribuzione dei percorsi e gli istanti di immissione, pur mantenendo la medesima struttura

generale descritta in Tabella 3.3; tali varianti vengono quindi alternate ciclicamente durante l’addestramento. Anche le configurazioni di validazione e test, su cui i modelli vengono valutati metodicamente, vengono generate con diversi *seed*, così da ottenere statistiche più robuste e informative sui risultati conseguiti.

In secondo luogo, per mettere alla prova la capacità del modello di adattarsi a situazioni locali di traffico differenti, è stata indotta un’asimmetria direzionale, designando una riga orizzontale della griglia come arteria principale (Est-Ovest), dunque con densità di traffico maggiore delle strade trasversali, mantenendo comunque invariato il volume totale di veicoli. Tale arteria viene posta su una riga diversa per ogni variante di addestramento, al fine di scongiurare l’ipotesi che le policy sviluppate siano dipendenti dalla posizione dell’incrocio, mentre viene mantenuta fissa sulla stessa riga nelle configurazioni di validazione e di test, in modo da garantire la comparabilità dei risultati.

Training, validation e test. La Tabella 3.4 raccoglie in un unico prospetto tutte le combinazioni di griglia, spaziatura e profilo di traffico introdotte in questo lavoro, con il relativo utilizzo nella pipeline sperimentale; le varianti seed di validazione e test sono omesse, poiché non introducono combinazioni distinte da quelle elencate. In tutto possiamo contare 11 configurazioni: 3 di training, 1 di validation e 7 di test.

Griglia	Spaziatura	Profilo di traffico	Utilizzo
4×4	100m	Complete (arteria riga 0)	Train
4×4	100m	Complete (arteria riga 1)	Train
4×4	100m	Complete (arteria riga 2)	Train
4×4	100m	Flat	Validation
4×4	100m	Peak	Test
4×4	200m	Flat	Test
4×4	200m	Peak	Test
5×5	100m	Flat	Test
5×5	100m	Peak	Test
6×6	100m	Flat	Test
6×6	100m	Peak	Test

Tabella 3.4: Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo.

3.2.5 Semplificazioni dichiarate

Per completezza, alcune semplificazioni dell’ambiente vanno dichiarate esplicitamente invece di lasciarle implicite nel codice:

- Il giallo è trattato come un prolungamento del verde, non come uno stato fisico distinto: una scelta dichiarata, non equivoca dopo l'introduzione del tutto-rosso reale discussa sopra.
- CITYFLOW non simula la fisica di collisione punto per punto tra veicoli di movimenti diversi all'interno del poligono dell'intersezione; la sicurezza della transizione di fase è garantita a livello di simulazione dei movimenti abilitati e disabilitati (nessun veicolo nuovo entra su un movimento rosso), non da un motore fisico di collisione.
- Il campo visivo a cutoff di distanza e il proxy di congestione basato sul conteggio di veicoli per corsia (non sulla velocità media, non disponibile per singola corsia in CITYFLOW) sono approssimazioni dichiarate, non un tentativo di modellare la fisica del traffico in ogni dettaglio.
- Tutte le topologie di questo lavoro sono griglie regolari (§3.2.4), con strade parallele od ortogonali e isolati della stessa lunghezza fisica all'interno di una data configurazione: una rete stradale reale, con incroci irregolari e distanze variabili tra loro, non è rappresentata. La scelta isola l'effetto della sola dimensione della griglia sulla generalizzazione dagli effetti di un'irregolarità geometrica che questo lavoro non intende studiare, non è un tentativo di riprodurre la topologia di una città esistente.
- L'unico tipo di traffico simulato è quello veicolare privato: nessun pedone, ciclista, mezzo pubblico o veicolo di soccorso compare nell'ambiente, e ogni veicolo generato condivide gli stessi parametri fisici (lunghezza, accelerazione massima, velocità massima) definiti nel file di flusso, indipendentemente dalla configurazione. Un ambiente multimodale, con partecipanti dal comportamento eterogeneo, resta fuori dallo scopo di questo lavoro (si veda anche la nota sulla scalabilità multimodale nel Capitolo 2).
- Il limite di velocità $v_{\text{urb}} = 11.1 \text{ m/s}$ (§3.1) è un'unica costante globale, identica per ogni corsia della rete e per ogni veicolo generato: non esiste una distinzione tra un'arteria a scorrimento più veloce e una strada locale, né tra tipologie di veicolo con velocità massime diverse.
- Ogni intersezione di questo lavoro è a quattro vie e dispone dello stesso insieme di $N_PHASES = 8$ fasi semaforiche (§3.2, Tabella 3.1), incluse le intersezioni ai margini della griglia con fisicamente meno di quattro strade collegate, per le quali le corsie mancanti sono modellate come corsie fittizie sempre a zero

veicoli: un incrocio a tre vie o a rotatoria, con un numero di fasi diverso da otto, non è rappresentato in nessuna configurazione di questo lavoro.

- Ogni corsia in ingresso è dedicata a un solo movimento (sinistra, diritto o destra, §3.2): un veicolo entra già nella corsia corrispondente alla propria prossima svolta, non deve mai cambiare corsia per raggiungerla. Il cambio di corsia non è permesso (`laneChange=False` nella configurazione CITYFLOW, già usato nel Capitolo 4, §4.2 per argomentare il determinismo bit-per-bit della simulazione): un veicolo resta sulla corsia di generazione per l'intero percorso, un'assunzione di comodo che evita di modellare la contrattazione tra veicoli per l'accesso a una corsia, non un tentativo di riprodurre il comportamento di guida reale.
- Ogni strada di questo lavoro è a doppio senso di marcia: la generazione parametrica del roadnet (§3.2.4) crea sempre due oggetti "strada" indipendenti, uno per direzione, per ogni coppia di intersezioni adiacenti, ciascuno con il proprio insieme di corsie. Non esiste un meccanismo per generare una strada a senso unico in nessuna configurazione di questo lavoro, una semplificazione che una rete stradale reale, dove i sensi unici sono comuni specialmente nei centri urbani, non condivide.

3.3 Baseline classiche

Fixed-Time. Cicla le 8 fasi in ordine fisso $(0, 1, \dots, 7, 0, \dots)$, cambiando fase a ogni passo di decisione (15s): non osserva mai lo stato della rete, non ha alcun parametro da configurare oltre all'ordine del ciclo, e non richiede training. La durata di ogni fase nel ciclo è deliberatamente identica a quella di ogni altro controllore di questo lavoro, RL o reattivo: un ciclo più lungo o più corto introdurrebbe un fattore di scala arbitrario nel confronto, rendendo impossibile distinguere un vantaggio dovuto alla strategia di controllo da uno dovuto solo a una cadenza diversa. Fixed-Time rappresenta, in questo lavoro, il limite superiore di travel time che qualunque strategia informata, reattiva o appresa, dovrebbe poter migliorare (§4).

Max-Pressure. Implementazione fedele al riferimento LibSignal (Varaiya, 2013), un controllore reattivo, senza parametri appresi e senza necessità di training: a ogni passo di decisione, per ogni intersezione, calcola la pressione di ciascuna delle 8 fasi e seleziona quella a pressione massima. La pressione di una fase ϕ è la somma, su tutti i movimenti (coppia corsia di ingresso/corsia di uscita) abilitati da ϕ , della

differenza tra veicoli sulla corsia di ingresso e veicoli sulla corsia di uscita, contati sull'intera corsia (nessun cutoff di visibilità):

$$P(i, \phi) = \sum_{(\ell_{\text{in}}, \ell_{\text{out}}) \in \text{movimenti}(\phi)} \left(n(\ell_{\text{in}}) - n(\ell_{\text{out}}) \right) \quad (3.5)$$

L'azione scelta è $\phi^*(i) = \arg\max_{\phi} P(i, \phi)$ (a parità di pressione, l'indice di fase più basso, per stabilità); nessun vincolo di equità o anti-starvation è applicato, a differenza dell'agente RL (§3.1), e nessuna maschera sulle azioni consecutive: MaxPressure può, in linea di principio, scegliere la stessa fase per un numero arbitrario di passi consecutivi, se quella resta la fase a pressione più alta. Alcune implementazioni di riferimento (incluso LibSignal) prevedono un tempo minimo di permanenza su una fase prima di poterla cambiare, necessario quando l'agente può essere interrogato a una cadenza più fine di quella minima di sicurezza; nell'ambiente di questo lavoro ogni decisione dura già `STEP_TIME = 15` secondi per costruzione (§3.2), quindi non esiste una cadenza più fine a cui l'agente venga interrogato, e questo vincolo aggiuntivo risulta già implicitamente soddisfatto senza bisogno di reimplementarlo esplicitamente.

3.4 Architettura MetaSTGAT

MetaSTGAT (Wang et al., 2022) elabora lo stato s_i^t di ogni intersezione attraverso cinque componenti in sequenza, prima di produrre i valori Q per le 8 azioni disponibili. La descrizione che segue è stata verificata riga per riga sul codice sorgente del modello (`src/models/`), non trascritta dalla sola notazione del paper: dove il paper è ambiguo (in particolare per Meta-GAT e Meta-LSTM, §3.4.3-3.4.4), è indicata esplicitamente quale delle interpretazioni possibili il codice implementa.

3.4.1 State Encoder duale

Lo stato $s_i^t \in \mathbb{R}^{d_0}$ ($d_0 = 32$ o 20 , §3.1) attraversa due MLP a due strati con la stessa architettura ma pesi e inizializzazioni indipendenti:

$$e_i^t = \text{ReLU}\left(W_{21} \text{ReLU}(W_{11}s_i^t + b_{11}) + b_{21}\right) \quad (3.6)$$

$$e_j^t = \text{ReLU}\left(W_{22} \text{ReLU}(W_{12}s_i^t + b_{12}) + b_{22}\right) \quad (3.7)$$

entrambi in $\mathbb{R}^{d_1}$ ($d_1 = 64$ di default). e_i^t (branch temporale) alimenta il Meta-LSTM (§3.4.4); e_j^t (branch spaziale) è la query di entrambi i moduli Meta-GAT (§3.4.3).

3.4.2 Meta-knowledge learner

Due MLP a due strati (stessa architettura dello state encoder, con un'attivazione tanh finale opzionale che normalizza l'embedding in $[-1, 1]$, disattivabile con il flag `-no-tanh-meta`) apprendono una rappresentazione compatta di caratteristiche locali dell'intersezione, usata a valle solo per generare i pesi di Meta-GAT e Meta-LSTM, mai come input diretto del Q-value:

$$\text{SMK}(i) = f_{\text{smk}}(x_i^{\text{smk}}) \in \mathbb{R}^{26 \rightarrow 64} \quad (3.8)$$

$$\text{TMK}(i) = f_{\text{tmk}}(x_i^{\text{tmk}}) \in \mathbb{R}^{25 \rightarrow 64} \quad (3.9)$$

Il vettore spaziale $x_i^{\text{smk}} \in \mathbb{R}^{26}$ concatena cinque gruppi di feature, nessuna delle quali entra anche nello stato principale s_i^t : pressione per corsia [12], frazione di vicini reali su `num_neighbors` [1], differenza tra la pressione media propria e quella di ciascun vicino reale [4, con 0 per gli eventuali slot senza vicino], asimmetria di pressione tra le corsie N/S e le corsie E/O dello stesso nodo [1] e pressione per fase [8], la stessa quantità usata da Max-Pressure (§3.3) per scegliere la propria azione, qui offerta al meta-learner come segnale invece che come regola di decisione diretta. La frazione di vicini reali sostituisce una feature di distanza dai vicini presente in una prima versione di questo lavoro: su reti a lunghezza di strada uniforme, come tutte quelle usate qui, la distanza euclidea normalizzata tra due vicini reali è quasi sempre pari a 1 per costruzione, una feature di fatto costante e quindi non informativa per un meta-learner. Il vettore temporale $x_i^{\text{tmk}} \in \mathbb{R}^{25}$ concatena la variazione della coda per corsia rispetto a qualche passo addietro [12], da quanti passi consecutivi è attiva la fase corrente, saturato e normalizzato in $[0, 1]$ [1], e la deviazione standard della coda per corsia sulla finestra recente [12]; sostituisce una versione precedente che includeva direttamente lo storico grezzo di n_i^t sugli ultimi passi, ridondante con la memoria che la componente ricorrente del modello (§3.4.4) mantiene già autonomamente nel proprio stato interno. La feature "location" menzionata nel paper originale per il TMK non è implementata in questo lavoro: un'omissione dichiarata, non una scelta motivata a posteriori (§3.4.6, punto C3).

3.4.3 Meta-GAT

Il meccanismo qui descritto si discosta dal GAT di [Veličković et al. \(2018\)](#) presentato nel Capitolo 2 su due fronti indipendenti, non uno solo. Il primo, già anticipato in quella sede, è la generazione dinamica dei pesi da parte del meta-learner al posto di una proiezione W fissa e condivisa. Il secondo, introdotto solo qui, riguarda la fun-

zione di compatibilità stessa: al posto della concatenazione additiva $\vec{a}^\top [Wh_i \| Wh_j]$ di GAT, Meta-GAT adotta Query, Key e Value esplicite e distinte, con un prodotto scalare scalato nello stile dell'attenzione multi-head di Vaswani et al. (2017) (già introdotta nel Capitolo 2) al posto del layer feedforward a un vettore $\vec{a}$. Il modello usa due istanze dello stesso layer Meta-GAT, con input e meta-embedding diversi:

- **Modulo CS** (spaziale puro): $Q = K = V = e_j^t$, meta-embedding = SMK(i), produce z_{CS} .
- **Modulo CST** (spazio-temporale): $Q = e_j^t$, $K = V = x_i^t$ (l'output del Meta-LSTM, §3.4.4), meta-embedding = TMK(i), produce z_{CST} .

Per ciascun modulo, con $H = 4$ teste di dimensione $d_h = d_1/H = 16$: il meta-embedding genera, per ogni nodo i e ogni testa h , una matrice $W_h^{(i)} \in \mathbb{R}^{d_h \times d_h}$ e uno scalare $b_h^{(i)}$ (stessa rete a due strati usata per SMK/TMK: un layer condiviso con ReLU seguito da due proiezioni lineari indipendenti per W e b). Per ogni arco ($u \rightarrow i$) del grafo (§3.1):

$$\varphi_h(i, u) = \frac{(W_h^{(i)} Q_h(i)) \cdot K_h(u) + b_h^{(i)}}{\sqrt{d_h}}, \quad \alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))} \quad (3.10)$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u), \quad z(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1} \quad (3.11)$$

Due punti dell'Equazione 3.10 risolvono un'ambiguità del paper (Eq. 14-17), che non specifica la dimensione di W né distingue $\sqrt{d_h}$ da $\sqrt{d_1}$: il codice applica $W_h^{(i)}$ alla query del nodo *ricevente* i (non del vicino u) prima del prodotto scalare con la key, e usa $\sqrt{d_h}$ (dimensione della singola testa) come fattore di scala, coerente con la formulazione standard dell'attenzione multi-head (Vaswani et al., 2017).

3.4.4 Meta-LSTM

Una rete a due strati genera, dal solo TMK(i), i pesi $W_g^{(i)} \in \mathbb{R}^{2d_1 \times d_1}$ e i bias $b_g^{(i)} \in \mathbb{R}^{d_1}$ per ciascuno dei quattro gate $g \in \{\text{forget, input, output, cell}\}$ di una cella LSTM altrimenti standard:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(\left(W_{\{f, \iota, o, g\}}^{(i)} \right)^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f, \iota, o, g\}}^{(i)} \right) \quad (3.12)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t \quad (3.13)$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (3.14)$$

con $[e_i^t \parallel h_i^{t-1}] \in \mathbb{R}^{2d_1}$ concatenazione dell’encoding corrente e dello stato nascosto precedente (ordine input-poi-hidden, coerente con la notazione del paper $\text{LSTM}(e_i^t, h_{t-1})$). L’output x_i^t (rinominato h_i^t per uniformità con la notazione LSTM standard) alimenta il modulo CST di Meta-GAT come K e V (§3.4.3). Il paper (Eq. 20) scrive $W_\Phi \in \mathbb{R}^{2D_h \times D_h}$ e $b_\Phi \in \mathbb{R}$ (un solo gate, bias scalare): il codice genera pesi e bias completi per tutti e quattro i gate, l’unica scelta compatibile con una LSTM funzionante (§3.4.6, categoria B).

3.4.5 Testa Q-value

I due embedding spaziali si concatenano e attraversano un singolo layer lineare, fedele all’Eq. 12 del paper:

$$q(s_i^t, \cdot) = W_q [z_{\text{CST}}(i) \parallel z_{\text{CS}}(i)] + b_q \in \mathbb{R}^8 \quad (3.15)$$

3.4.6 Dal paper al modello Pro: differenze motivate

Il modello Pro di questo lavoro non è un’architettura diversa dal paper, ma la stessa architettura con una serie di differenze puntuali rispetto alla Section 4.3 e all’Algorithm 1 originali, ciascuna motivata singolarmente e disattivabile indipendentemente tramite il sistema di ablation a preset (§3.5). Le differenze si dividono in tre categorie, a seconda di quanto la loro superiorità sia già stabilita o resti da verificare sperimentalmente.

A. Miglioramenti con motivazione tecnica solida.

- A1. *w_i^t nello stato* (§3.1): un segnale più informativo del solo conteggio veicoli, a parità di coda un’attesa di 90s è più urgente di una di 5s.
- A2. *Maschera azioni invalide*: previene il collasso su una singola fase (verde fisso), osservato come comportamento degenerativo nelle fasi iniziali di training senza vincolo.
- A3. *Double DQN*: riduce il bias di sovrastima del valore Q rispetto al DQN standard (van Hasselt et al., 2016).
- A4. *BPTT + burn-in* (sequenze $L = 4$, burn-in 2, R2D2-style Kapturowski et al., 2019): corregge l’hidden state staleness dello stato LSTM tra raccolta dati e training.

- A5. *Huber loss + pesi IS da PER*: più robusta agli outlier della MSE; il campionamento prioritario concentra il training sulle transizioni a errore TD più alto.
- A6. *Soft update* (Polyak, $\tau = 0.01$): target network più stabile di un hard-copy periodico.
- A7. *Gradient clipping* ($\|\nabla\| \leq 1.0$): stabilità numerica necessaria con un componente ricorrente nel grafo di computazione.
- A8. *tanh finale nei meta-learner*: normalizza l’embedding generato in $[-1, 1]$ invece di lasciarlo libero.
- A9. *Cutoff di visibilità* ($\approx 144\text{m}$): realismo fisico, non un numero arbitrario (§3.2).
- A10. *Buffer PER a 4.800 sequenze* (vs 10.000 transizioni piatte del paper): calibrato deliberatamente sulla durata dell’episodio (≈ 40 episodi di storia con episodi da 1800s), non un ridimensionamento subito: la metrica di confronto corretta con il paper è la copertura in episodi, non il numero grezzo di transizioni.

B. Disambiguazioni di un paper sotto-specificato. La formula di attenzione di Meta-GAT (Eq. 14-17 del paper, §3.4.3) e le dimensioni dei pesi di Meta-LSTM (Eq. 20, §3.4.4) non sono deviazioni arbitrarie: il paper stesso lascia aperta più di un’interpretazione, e le scelte implementative descritte sopra sono quelle compatibili con un’architettura funzionante, non una tra più alternative egualmente valide.

C. Scelte da presentare con cautela. Un’ultima categoria di differenze non è ancora, a questo stadio del lavoro, un miglioramento dimostrato: nessun preset di ablation diverso da `pro` e `temporal` (§3.5) è stato allenato e testato al momento della stesura di questo capitolo.

- C1. *Ricompensa riscritta* (Eq. 3.3 vs Eq. 3.4): la motivazione teorica è che una ricompensa di pura pressione può essere instabile, e che promuovere esplicitamente il throughput e penalizzare le attese prolungate produce un comportamento più bilanciato; il suo effetto netto su travel time e throughput, a parità di resto dell’architettura, è discusso nel Capitolo 4 alla luce dei risultati dell’ablation su `reward_mode`, non dato per acquisito qui.
- C2. *Numero di episodi ed epsilon decay*: il vincolo di risorse (un solo processo, mai i 3 processi paralleli del paper) va dichiarato onestamente, ma non è di per sé un limite subito: il meccanismo di selezione del modello finale (§3.5) mostra

che il training raggiunge il suo miglior risultato ben prima di esaurire il budget di episodi assegnato.

C3. *Feature "location" assente nel TMK*: omissione dichiarata, senza una motivazione cercata a posteriori.

3.5 Procedura di training e differenze dal paper originale

Il modello è addestrato con Double DQN (A3) su sequenze campionate da un replay buffer prioritizzato episodico (A10), con burn-in e BPTT (A4), Huber loss pesata dagli importance-sampling weight di PER (A5), soft update della target network (A6) e gradient clipping (A7); l'ottimizzatore è RMSprop, non Adam, per fedeltà alla Section 5.1 del paper originale: l'unica scelta architetturale/di training che questo lavoro non ha mai messo in discussione né reso configurabile via ablation, a differenza di ogni altra voce di questa sezione.

Iperparametri usati per il modello Pro (identici su tutte le varianti dello sweep su α): *learning rate* 10^{-3} , $\gamma = 0.85$, $d_1 = 64$, $H = 4$ teste, `num_neighbors` = 4, ε da 0.9 a 0.01 con decadimento esponenziale, batch size 20, buffer PER a 4.800 sequenze di lunghezza $L = 4$ con burn-in 2 (A4, A10). Ogni run di questo lavoro usa 100 episodi su un solo processo, contro i 200 episodi su 3 processi paralleli del paper (C2): il divario nel numero di episodi totali di raccolta dati è reale e dichiarato, ma, per la ragione discussa in C2, non equivale a un training interrotto prematuramente.

Ciclo di training. Prima dell'inizio del training vero e proprio, 10 episodi di *warm-up* (disattivabili) raccolgono transizioni con $\varepsilon = 1$ (azioni interamente casuali, non loggate come training vero) con il solo scopo di riempire il replay buffer fino a una soglia minima di 1.000 transizioni, sotto la quale il campionamento per l'aggiornamento dei pesi non è ancora avviato; con più configurazioni di training in ciclo, anche il warm-up alterna tra le varianti, così il buffer iniziale non proviene da una sola di esse. Nel training vero e proprio, ogni episodio appartiene a una sola variante di configurazione (alternata deterministicamente per numero di episodio, §3.2.4), al termine del quale l'agente esegue 100 aggiornamenti del gradiente: un numero fisso, indipendente dal numero di passi ambientali dell'episodio (120 con episodi da 1800 secondi), ereditato dal paper originale come iperparametro a sé stante, non ricalibrato quando la durata degli episodi è cambiata nel corso di questo lavoro. Il valore di ε decade una volta per episodio verso il suo minimo, raggiunto al 60% del nume-

ro totale di episodi pianificati per quel run (il tasso di decadimento non è quindi un numero fisso in assoluto, ma viene ricalcolato in funzione del numero effettivo di episodi richiesto): usare ε -greedy come strategia di esplorazione (§2.1) significa che, a ogni passo di decisione, l’agente sceglie un’azione casuale con probabilità ε e altrimenti l’azione a valore Q massimo secondo la stima corrente.

Oltre a questo decadimento continuo, un episodio ogni 10 viene eseguito interamente a $\varepsilon = 1$ anche dopo che il valore minimo di ε è stato raggiunto (*esplorazione ciclica*, disattivabile), senza contribuire all’aggiornamento dei pesi in quello specifico episodio: l’obiettivo è continuare a inserire nel buffer esperienza che la politica ormai quasi deterministica non visiterebbe più spontaneamente, invece di lasciare che il replay buffer converga verso le sole traiettorie che la politica appresa preferisce. Questo lavoro ha esplicitamente valutato, e poi scartato, l’ipotesi che disattivare questo meccanismo una volta raggiunto il minimo di ε migliorasse la stabilità del training: un episodio interamente casuale a politica ormai quasi deterministica inserisce nel buffer transizioni potenzialmente fuori distribuzione con priorità massima di default, e un picco ripetuto nella loss subito dopo ogni occorrenza tardiva di questo meccanismo sembrava inizialmente confermare un effetto negativo; controllando però l’effetto per la variante di training attiva in quell’episodio (che cambia comunque a ogni episodio, indipendentemente dall’esplorazione ciclica), l’impatto sull’episodio successivo è risultato inconcludente, non sistematicamente negativo. Non abbastanza per modificare un meccanismo esistente e deliberatamente motivato: il comportamento originale è stato mantenuto.

Il sistema di ablation a preset (`pro/paper/ environment/temporal/rl_core/ replay_stability`) rende ciascuna delle differenze A1-A10, B e C1-C3 sopra individualmente disattivabile da riga di comando: il preset `paper` le disattiva tutte simultaneamente, riproducendo la procedura originale con lo stesso codice usato per il modello Pro. Il Capitolo 4 riporta i risultati sperimentali ottenuti, incluso lo studio di ablation costruito su questo stesso sistema di preset.

Capitolo 4

Confronto

4.1 Metriche di valutazione

Il reward di training (Equazioni 3.3-3.4) non è comparabile tra modalità **custom** e **paper**, per cui ogni confronto in questo capitolo usa esclusivamente metriche oggettive, calcolate allo stesso modo indipendentemente da come un modello è stato addestrato.

Travel time. Per ogni veicolo v , il tempo di viaggio è

$$tt_v = \begin{cases} t_{\text{arrivo}}(v) - t_{\text{spawn}}(v) & \text{se } v \text{ è arrivato entro la fine dell'episodio} \\ t_{\text{now}} - t_{\text{spawn}}(v) & \text{altrimenti (un lower bound del suo vero valore)} \end{cases} \quad (4.1)$$

La metrica ufficiale di questo lavoro, $\overline{tt}$, è la media di tt_v calcolata sull'intera popolazione di veicoli generati nell'episodio, non solo su quelli arrivati: include cioè deliberatamente i veicoli che non hanno completato il percorso entro la fine della simulazione. La ragione è evitare un bias di sopravvivenza concreto: un modello che congestiona la rete e lascia arrivare solo una minoranza di veicoli su percorsi rimasti liberi otterrebbe, calcolando la media solo sui veicoli arrivati, un risultato buono calcolato su un campione piccolo e non rappresentativo del comportamento reale della rete; nel caso limite in cui nessun veicolo arrivasse affatto, quella stessa metrica restituirebbe un travel time di 0 secondi, il punteggio migliore possibile per il comportamento peggiore immaginabile. Il travel time dei soli veicoli arrivati ($\overline{tt}_{\text{arr}}$) è riportato in ogni tabella di questo capitolo come dato informativo aggiuntivo, mai come criterio con cui scegliere tra modelli. Una terza variante, il travel time nativo calcolato direttamente dal motore CITYFLOW (che adotta anch'esso la convenzione "solo veicoli arrivati"), non è riportata in questo capitolo: serve solo per un confronto

diretto con un numero pubblicato in letteratura (il paper originale riporta un travel time di 430-470 secondi su una rete sintetica 4×4 , non direttamente comparabile con i valori di questo lavoro data la ricompensa e la procedura di training diverse, §3.4.6), non per confrontare i modelli di questo lavoro tra loro.

Coda del travel time. Oltre alla media, si riportano il massimo ($tt_{\max}$), la deviazione standard e i percentili $p_{90}/p_{95}/p_{99}$ del travel time dei veicoli arrivati: una media bassa può coesistere con una coda lunga di veicoli fortemente penalizzati, e nessuna delle due statistiche da sola distinguerebbe questo caso da una distribuzione uniformemente buona.

Throughput. Veicoli che hanno completato il percorso entro la fine dell'episodio, riportato sia in valore assoluto sia come percentuale sul totale dei veicoli generati (indicata $\%_{\text{compl}}$ nelle tabelle di questo capitolo). A differenza del travel time, il throughput non richiede alcuna convenzione sui veicoli non arrivati: un veicolo o ha completato il percorso o non lo ha fatto, senza casi intermedi da trattare.

Percentuali di scelta delle fasi. Per ciascuna delle 8 fasi (Tabella 3.1), la frazione di passi di decisione, su tutte le intersezioni e l'intero episodio, in cui quella fase è stata scelta. Non è di per sé una metrica di prestazione, nel senso che nessun valore è "migliore" in assoluto: è piuttosto un diagnostico della strategia appresa o applicata da un controllore. Un controllore che collassa sistematicamente su un piccolo sottoinsieme di fasi, ad esempio, sta probabilmente reagendo a uno squilibrio strutturale del traffico (una direzione più trafficata delle altre) piuttosto che alternare in modo bilanciato; MaxPressure, per la sua logica greedy sulla pressione istantanea, tende di fatto a preferire marcatamente le fasi che servono i movimenti "diritto" rispetto a quelle che servono le sole svolte a sinistra, dato che i primi accumulano pressione più rapidamente su una rete con volumi tipici.

Equità (*fairness*) come obiettivo distinto dall'efficienza media. In letteratura sul controllo semaforico, l'equità indica che tutti gli utenti della strada ricevano un accesso equo alle risorse dell'intersezione, evitando che alcuni veicoli o alcune direzioni sperimentino attese eccessive rispetto ad altri (Xiao et al., 2025). È un obiettivo concettualmente distinto dall'efficienza aggregata di rete (travel time medio, throughput): un controllore può massimizzare il flusso complessivo e allo stesso tempo lasciare una minoranza di veicoli sistematicamente penalizzata, se la sua regola di decisione non include alcun meccanismo che penalizzi esplicitamente le attese prolungate. È esattamente il caso di Max-Pressure (§3.3): massimamente

stabilizzante per la rete nel suo complesso (Varaiya, 2013), ma privo di qualunque garanzia sulla distribuzione del servizio tra le direzioni, dato che sceglie sempre la fase a pressione istantanea più alta senza mai considerare da quanto tempo attende la direzione esclusa.

Alcuni metodi recenti in letteratura affrontano il problema penalizzando esplicitamente le fasi responsabili di attese eccessive, tipicamente solo oltre una soglia, per non sacrificare le prestazioni medie quando non è necessario: FELight, ad esempio, introduce una metrica di equità basata sui tempi di attesa dei veicoli e penalizza selettivamente le fasi con ritardi eccessivi, riportando una riduzione delle attese prolungate su direzioni specifiche senza compromettere l'efficienza complessiva della rete (Xiao et al., 2025). Questo lavoro persegue lo stesso obiettivo con un meccanismo diverso: il termine anti-starvation della ricompensa *custom* (Equazione 3.3) penalizza MaxRedWait_i^t , l'attesa massima osservata su una corsia non servita dalla fase corrente, pesata dall'iperparametro α — concettualmente lo stesso principio (penalizzare la coda peggiore, non solo ottimizzare il throughput aggregato), applicato però a ogni passo di decisione per ogni intersezione, non a un sottoinsieme di fasi identificate a priori. È il motivo per cui questo lavoro accetta, entro certi limiti, un travel time medio peggiore pur di ottenere un'attesa massima più contenuta (§4.4): a parità di rete, un controllore che fa aspettare pochi veicoli molto a lungo non è preferibile a uno leggermente più lento in media ma che non lascia mai nessuno bloccato a tempo indefinito.

Per misurare quanto questo meccanismo funzioni nella pratica, questo lavoro non si affida al solo reward (che dipende dalla parametrizzazione di α e non è confrontabile tra `reward_mode` diversi, si veda l'apertura di questa sezione) ma a una metrica puramente osservazionale, calcolata identicamente per ogni controllore indipendentemente da come è stato addestrato o se non è stato addestrato affatto (Fixed-Time, Max-Pressure):

Equità direzionale. Per ogni intersezione, la massima attesa storica *consecutiva* osservata su una corsia del gruppo N/S o del gruppo W/E (§3.1): consecutiva nel senso che il contatore si azzerava non appena il veicolo interessato si muove, non è quindi un'attesa cumulativa sull'intero viaggio del veicolo, ma la durata del più lungo singolo stallo mai osservato su una corsia di quel gruppo, in qualunque punto della rete. Il gruppo N/S raccoglie le corsie percorse da chi attraversa l'incrocio in direzione verticale (arrivando da Nord o da Sud), il gruppo W/E quelle percorse da chi lo attraversa in direzione orizzontale: è una classificazione per corsia fisica di approccio all'incrocio in un dato istante, non un'origine o una destinazione del

viaggio del veicolo. Il valore ufficiale, `wait_maxNS` e `wait_maxEW`, è il massimo di questa quantità su tutte le intersezioni della rete, e include gli stalli ancora in corso al termine dell’episodio (censura a destra), per lo stesso principio di non-esclusione del caso peggiore applicato al travel time: un veicolo fermo nello stesso punto dal momento in cui si è bloccato fino all’ultimo istante simulato non fa mai scattare, per definizione, un nuovo record *dopo* la fine dell’episodio, e una metrica che considerasse solo gli stalli conclusi lo farebbe sparire, premiando indirettamente chi lascia un veicolo bloccato per sempre invece di penalizzarlo come merita. Il caso non è ipotetico: durante lo sviluppo di questa metrica, un valore di `wait_maxNS = 1125` secondi riportato da MaxPressure è stato verificato ricostruendo la traiettoria del veicolo responsabile direttamente dal replay grezzo della simulazione (posizione per veicolo, secondo per secondo, un canale indipendente da qualunque contabilità interna del codice): il veicolo risultava fermo nello stesso identico punto dal secondo 679 fino al secondo 1799, l’ultimo istante simulato dell’intero episodio, uno stallo la cui vera durata resta sconosciuta perché la simulazione si è fermata, non perché il veicolo sia stato servito. Questa metrica non compare nella ricompensa di alcun modello: è puramente osservazionale, pensata per verificare se il termine anti-starvation della ricompensa `custom` (Equazione 3.3) produce un vantaggio reale rispetto a MaxPressure, che non ha alcuna nozione di equità, né di un vincolo simile alla maschera sulle azioni consecutive che regola invece l’agente RL (§3.1): i due meccanismi restano concettualmente indipendenti, e il rispetto dell’uno non implica il rispetto dell’altro. Una variante di questa metrica, che considera solo gli stalli conclusi entro la fine dell’episodio (esclude cioè la censura a destra discussa sopra), è calcolata e riportata come colonna secondaria in ogni tabella di riepilogo, ma non è mai usata come criterio ufficiale di confronto in questo capitolo, per lo stesso principio di non-esclusione del caso peggiore.

4.2 Protocollo sperimentale

Training, validazione, test. Ogni modello RL è addestrato in ciclo su tre varianti a seed diverso della stessa configurazione base (`config_4x4_100m_train`, `_s2`, `_s3`), con l’arteria Est-Ovest su una riga diversa in ciascuna (righe 1, 2, 0 rispettivamente, §3.2): un episodio di training appartiene sempre a una sola variante, alternata deterministicamente per numero di episodio. A fine training, due checkpoint candidati, l’ultimo episodio (`final_model.pth`) e l’episodio con il miglior travel time mai osservato durante il training (`best_model.pt`, non necessariamente l’ultimo), sono valutati per un episodio ciascuno su una configurazione di validazione

indipendente, `config_4x4_100m_6k_flat`, mai usata né in training né nel confronto finale tra modelli; il vincitore diventa `selected_model.pth`, il checkpoint cercato con priorità massima da ogni valutazione successiva (in sua assenza, la ricerca ripiega su `final_model.pth`, poi su `best_model.pt`, poi sul checkpoint periodico più recente disponibile). Ogni valutazione, sia in questa fase di selezione sia nella batteria di test descritta sotto, avviene a $\varepsilon = 0$: nessuna esplorazione residua, solo la politica appresa allo stato puro, dato che introdurre azioni casuali in fase di valutazione misurerebbe la qualità di una politica che non è quella che si intende effettivamente confrontare.

Batteria di test. Ogni modello, RL o baseline, è valutato sulle stesse 10 configurazioni: le 3 varianti di training stesse (per misurare la prestazione in-distribuzione) più 7 configurazioni mai viste in training (4x4 a 200m invece di 100m, 5×5 e 6×6 a densità sia *flat* sia *peak*), usate per misurare la generalizzazione a topologie e densità di traffico diverse. Testare ogni modello su entrambi i gruppi, non solo sulle configurazioni di generalizzazione, è la prassi adottata in questo lavoro dopo averla resa parte dell'orchestrazione standard (`main.py`).

Determinismo della simulazione. Con flusso di traffico e seed fissati nel file di configurazione e `laneChange=False`, CITYFLOW è deterministico bit-per-bit: a parità di modello e configurazione, ogni valutazione produce esattamente la stessa traiettoria. La valutazione standard usa quindi un solo episodio per configurazione (`n_eval=1`): ripeterlo non ridurrebbe alcuna varianza reale. Questo ha una conseguenza sull'interpretazione dei risultati di questo capitolo che va dichiarata esplicitamente: un singolo numero riportato per coppia (modello, configurazione) è l'esito esatto di un'unica realizzazione deterministica, non una media campionaria con un proprio errore standard. Le differenze tra modelli sullo stesso file di configurazione sono quindi confronti esatti, non stime soggette a rumore di campionamento; resta però aperta la domanda di quanto un singolo risultato varierebbe se la stessa configurazione fosse ricampionata con un seed diverso, cioè con un'altra realizzazione plausibile della stessa distribuzione di traffico.

Seed multipli sulle configurazioni di test. Per rispondere a questa domanda, ciascuna delle 7 configurazioni di test (non le 3 di training, già esse stesse varianti seed della stessa distribuzione, §3.2.4) è stata affiancata da fino a 4 varianti aggiuntive, generate con lo stesso metodo "sostanza-preservante" già usato per le configurazioni di validazione (§3.2.4): stesso profilo di densità, topologia e arteria, seed diverso per il rumore su pesi delle rotte e istanti di spawn. Per ogni modello e ogni

configurazione di test, il travel time medio riportato in questo capitolo è quindi, dove disponibile, la media su un massimo di 5 realizzazioni indipendenti (il seed originale più fino a 4 varianti), con la deviazione standard corrispondente riportata come barra d'errore nei confronti grafici. Il calcolo, condotto sull'intero parco di modelli ufficiali di questo lavoro ($19 \text{ modelli} \times 7 \text{ configurazioni}$, 133 coppie), permette due osservazioni:

- Il seed singolo usato inizialmente per ogni configurazione di test era leggermente pessimista rispetto alla media delle realizzazioni plausibili: passando a 5 seed, il travel time medio scende in 102 coppie su 133 (77%), con uno scostamento medio di -1.2% . Lo spostamento è piccolo e va nella stessa direzione per la maggior parte dei modelli, per cui il ranking relativo tra modelli non ne risulta capovolto nei confronti di questo capitolo, ma i confronti già stretti (pochi secondi di differenza) vanno letti con questo margine in mente.
- La variabilità tra seed non è uniforme tra le configurazioni: la deviazione standard relativa dello scostamento varia da un minimo dello 0.7% (`config_4x4_200m_6k_peak`) a un massimo del 2.4% (`config_5x5_100m_9.4k_flat`), quest'ultima la configurazione dove un singolo seed rischia maggiormente di non essere rappresentativo. Il confronto tra la stima a 3 e a 5 seed mostra inoltre un segnale di convergenza: lo scostamento medio tra le due medie è solo -0.27% (contro l' -1.2% tra 1 e 5 seed), indicando che gran parte del guadagno informativo si ottiene già passando da 1 a 3 seed, con rendimento marginale decrescente oltre quella soglia.

Le tre varianti di training (seed distinti sulla stessa distribuzione di traffico) restano lo strumento con cui questo lavoro approssima la stessa domanda sul lato training, per le config di training stesse, non ricampionate allo stesso modo delle 7 di test.

Sistema di ablation a preset. Ogni preset (`-ablation`) imposta un gruppo coerente di flag, isolando un sottosistema alla volta rispetto al modello Pro:

Per la riproducibilità: ogni modello di questo capitolo è ottenuto con lo stesso comando, a parità di configurazioni di training e di seed, cambiando solo il preset (e, per lo sweep descritto in §4.3, il peso α della ricompensa):

Listing 4.1: Comando di training, modello Pro con $\alpha = 0.2$.

```
python scripts/train.py \
    --config configs/config_4x4_100m_train1.json \
    configs/config_4x4_100m_train2.json \
```

Tabella 4.1: Preset di ablation e sottosistema isolato da ciascuno.

Preset	Reward	Cosa isola rispetto a Pro
pro	custom	(nessuno: modello di riferimento)
paper	paper	Replica fedele di Wang et al. (2022)
environment	paper	Ingegnerizzazione dell'MDP (visibilità, w_i^t , maschera)
temporal	custom	BPTT + burn-in sulla Meta-LSTM
rl_core	custom	Double DQN
replay_stability	custom	PER, Huber, soft update, grad clipping, warm-up, esplorazione ciclica

```
configs/config_4x4_100m_train3.json \
--model MetaSTGAT --episodes 100 --seed 42 \
--ablation pro --alpha 0.2 \
--output results/metastgat_pro_0.2
```

4.3 Risultati: baseline classiche e peso anti-starvation

Questa sezione confronta sei controllori: le due baseline classiche (§3.3), tre varianti del modello Pro che differiscono solo per il peso α della penalità anti-starvation nella ricompensa custom (Equazione 3.3), e `ablation_temporal` (discusso separatamente in §4.5, dove il confronto rilevante è con Pro a parità di α). $\alpha = 0.1$ e i quattro preset di ablation restanti (`paper`, `environment`, `rl_core`, `replay_stability`) sono in fase di addestramento al momento della stesura di questo capitolo: i risultati che seguono si limitano ai modelli già valutati sulla batteria di test completa. Per Fixed-Time non sono disponibili $tt_{\max}$ e le statistiche di equità direzionale (calcolate solo a partire dall'11/9/2026, dopo che questo modello era già stato valutato): le colonne corrispondenti sono segnate "n.d.".

Tabella 4.2: Travel time medio, massimo e percentuale di completamento, mediati sulle 3 configurazioni di training (in-distribuzione).

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	%compl
Fixed-Time	339.3	n.d.	84.9
MaxPressure	161.9	1070.0	96.6
MetaSTGAT Pro ($\alpha = 0.5$)	205.5	800.0	93.9
MetaSTGAT Pro ($\alpha = 0.2$)	194.0	745.0	94.9
MetaSTGAT Pro ($\alpha = 0.0$)	211.2	1350.0	91.5

Tabella 4.3: Come Tabella 4.2, mediata sulle 7 configurazioni di test (generalizzazione a topologia e densità mai viste in training).

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	% _{compl}
Fixed-Time	463.9	n.d.	79.8
MaxPressure	239.2	1635.0	88.5
MetaSTGAT Pro ($\alpha = 0.5$)	295.2	1142.1	86.8
MetaSTGAT Pro ($\alpha = 0.2$)	280.8	1103.6	87.6
MetaSTGAT Pro ($\alpha = 0.0$)	311.75	1607.1	84.6

Fixed-Time è, come atteso, il controllore peggiore su ogni colonna in entrambi i regimi: non osservando mai lo stato della rete, stabilisce il limite superiore di travel time che qualunque strategia informata (reattiva o appresa) dovrebbe poter migliorare. Tutti gli altri controllori lo battono nettamente (161.9-311.75s contro 339.3-463.9s), confermando che anche la regola più semplice tra quelle reattive (MaxPressure) coglie un segnale reale nello stato della rete.

MaxPressure resta il modello con il travel time medio più basso in entrambi i regimi, coerente con la sua ottimalità di throughput a livello di rete (Varaiya, 2013): nessuna delle varianti RL di questo lavoro la supera su questa singola metrica. All'interno dello sweep su α , $\alpha = 0.2$ ottiene il travel time medio più basso e, in entrambi i regimi, anche il $tt_{\max}$ più basso tra tutti i cinque modelli confrontati, MaxPressure incluso: una penalità anti-starvation moderata non solo non peggiora il caso medio rispetto a $\alpha = 0.5$, ma produce la coda più corta osservata in questo esperimento. $\alpha = 0.0$ (nessuna penalità) è invece la configurazione peggiore tra le tre varianti Pro su ogni colonna delle due tabelle: rimuovere il termine anti-starvation non libera il modello per ottimizzare meglio il throughput medio, lo peggiora anche su quello. Questo risultato è discusso più a fondo, insieme all'equità direzionale che lo motiva, in §4.4.

Tutti i modelli degradano passando dalle configurazioni di training a quelle di test, MaxPressure incluso (161.9→239.2s): una parte di questo divario riflette la maggiore difficoltà intrinseca delle configurazioni di generalizzazione (topologie più grandi, densità di picco), non solo un limite di generalizzazione specifico del controllo appreso.

4.4 Equità direzionale: il ruolo di α

Il divario è netto. Con la penalità anti-starvation al suo valore di riferimento ($\alpha = 0.5$), l'attesa massima direzionale è 2-3 volte più bassa di MaxPressure in entrambi

Tabella 4.4: Attesa massima direzionale (N/S e W/E), mediata sulle configurazioni di training e di test rispettivamente.

Modello	Training (s)		Test (s)	
	N/S	W/E	N/S	W/E
MaxPressure	755.0	825.0	1407.9	1493.6
MetaSTGAT Pro ($\alpha = 0.5$)	300.0	345.0	591.4	546.4
MetaSTGAT Pro ($\alpha = 0.2$)	455.0	450.0	516.4	565.7
MetaSTGAT Pro ($\alpha = 0.0$)	1275.0	1130.0	1530.0	1133.6

i regimi; a $\alpha = 0.2$ resta comunque molto più bassa (516-566s contro 1408-1494s in test), pur essendo leggermente peggiore di $\alpha = 0.5$ su tre delle quattro colonne. A $\alpha = 0.0$, l'equità direzionale collassa verso valori vicini a quelli di MaxPressure (1275-1530s in training e nella direzione N/S del test, paragonabile o peggiore su W/E in training): esattamente il comportamento atteso rimuovendo dalla ricompensa l'unico termine che penalizza le attese prolungate sulle corsie rosse.

Il punto di correttezza sperimentale già anticipato in §4.2 si applica direttamente qui: ciascuno di questi numeri è l'esito di un'unica realizzazione deterministica per configurazione, mediata su un piccolo numero di configurazioni (3 o 7). L'effetto di α sull'equità direzionale è ampio e consistente in direzione su tutte le configurazioni osservate (non un singolo caso isolato), il che lo rende un risultato qualitativamente solido; non è tuttavia accompagnato, per questo specifico sweep ($\alpha \in \{0.5, 0.2, 0.0\}$), da un test di significatività statistica in senso classico, che richiederebbe più istanze indipendenti della stessa configurazione. Il protocollo a seed multipli descritto in §4.2 (paragrafo "Seed multipli sulle configurazioni di test"), che colma esattamente questa lacuna, è stato introdotto dopo questo sweep e applicato allo sweep ufficiale successivo ($\alpha \in \{0.08, 0.04, 0.02, 0.00\}$, non ancora riportato in questo capitolo alla data di stesura): i numeri di questa sezione restano quindi, alla data di stesura, basati su singola realizzazione.

Il fatto che $\alpha = 0.0$ sia anche il peggiore, non il migliore, su travel time medio e massimo (Tabelle 4.2-4.3) esclude la spiegazione più semplice di un compromesso equità-contro-throughput: in questi dati, il termine anti-starvation non compra equità al prezzo del throughput, sembra piuttosto aiutare entrambi contemporaneamente, quantomeno nell'intervallo $\alpha \in [0.2, 0.5]$ esplorato qui. Una possibile lettura è che una rete dove nessuna corsia resta bloccata a tempo indefinito fluisce meglio nel suo complesso, non solo per i veicoli sulla corsia sfavorita: la verifica di questa ipotesi richiederebbe un'analisi per-corsia o per-fase che esula dallo scopo di questo capitolo.

4.5 Studio di ablation: risultati preliminari

Al momento della stesura di questo capitolo è disponibile un solo confronto completo del sistema di ablation a preset (Tabella 4.1): `temporal` (BPTT e burn-in sulla Meta-LSTM disattivati) contro Pro allo stesso $\alpha = 0.5$, l'unica variabile che li distingue. I quattro preset restanti (`paper`, `environment`, `rl_core`, `replay_stability`) sono in addestramento; questa sezione va integrata con i loro risultati non appena disponibili, senza modificarne l'impostazione.

Tabella 4.5: MetaSTGAT Pro ($\alpha = 0.5$) contro `ablation_temporal` (BPTT e burn-in disattivati), stesso α .

Modello	Training		Test	
	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\overline{tt}$ (s)	$tt_{\max}$ (s)
Pro ($\alpha = 0.5$)	205.5	800.0	295.2	1142.1
<code>ablation_temporal</code>	220.1	865.0	350.2	1375.7

Disattivare BPTT e burn-in peggiora il travel time medio su entrambi i regimi (+7.1% in training, +18.6% in test) e allarga anche la coda ($tt_{\max}$ +8.1% e +20.5% rispettivamente). Il fatto che il peggioramento relativo sia maggiore in test che in training è un indizio, non una prova data la numerosità limitata, che il disallineamento dello stato ricorrente corretto da BPTT+burn-in (§2.1, *hidden state staleness*) non sia solo un problema di adattamento alla distribuzione di training, ma pesi anche sulla capacità del modello di generalizzare a topologie mai viste: un'ipotesi coerente con la motivazione originale del meccanismo (Kapturowski et al., 2019), che richiederebbe tuttavia il confronto con gli altri preset (in particolare `rl_core` e `replay_stability`) per essere isolata con maggiore sicurezza dal contributo di Double DQN, PER e delle altre differenze architetturali che `temporal` lascia invece attive.

Bibliografia

- Andrea Cini, Ivan Marisca, Daniele Zambon, and Cesare Alippi. Graph deep learning for time series forecasting. *ACM Computing Surveys*, 57(12):321, 2025. doi: 10.1145/3742784.
- Zhiyuan Hu, Bowen Liu, Jianming Hu, Zihang Wang, Lihui Peng, and Yi Zhang. Mpmnlight: A self-attention enhanced message passing graph neural network for multi-intersection traffic signal control. *Electronics*, 15(8):1655, 2026. doi: 10.3390/electronics15081655.
- Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018. doi: 10.1109/ITSC.2018.8569938.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

- Chao Sun, Yuhao Yang, Jiacheng Li, Weiyi Fang, and Peng Zhang. A multi-agent regional traffic signal control system integrating traffic flow prediction and graph attention networks. *Systems*, 14(1):47, 2026. doi: 10.3390/systems14010047.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Alessandro Trenta, Alessio Gravina, and Davide Bacciu. Sonar: Long-range graph propagation through information waves. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Jing Wang, Xiaopeng Wang, Yang Mo, Chao Huang, Yanan Li, Guanghe Li, and Hao Zhang. Attention-based spatio-temporal graph convolutional networks-enhanced deep reinforcement learning for adaptive traffic signal control in urban traffic networks. *Scientific Reports*, 2026. doi: 10.1038/s41598-026-61371-3.
- Min Wang, Libing Wu, Man Li, Dan Wu, Xiaochuan Shi, and Chao Ma. Meta-learning based spatial-temporal graph attention network for traffic signal control. *Knowledge-Based Systems*, 250:109166, 2022. doi: 10.1016/j.knosys.2022.109166.
- Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019. doi: 10.1145/3357384.3357902.

Feng Xiao, Jiaming Lu, Lu Li, Wenwen Tu, and Chaojing Li. Advances in reinforcement learning for traffic signal control: a review of recent progress. *Intelligent Transportation Infrastructure*, 2025:liaf009, 2025. doi: 10.1093/iti/liaf009.

Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haili Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW)*, 2019. doi: 10.1145/3308558.3314139.